Lecture 5

CME 295: Transformers & Large Language Models





Afshine Amidi & Shervine Amidi



Stanford University

ICME



Transformers & Large Language Models

Motivation

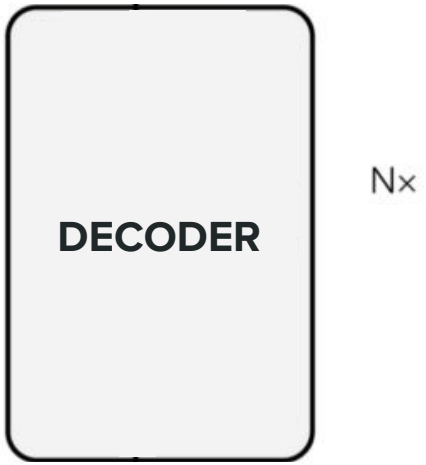
Flash attention

Distributed training

Inference optimizations

Discussion

Context



Model size: billions (or even trillions) of parameters

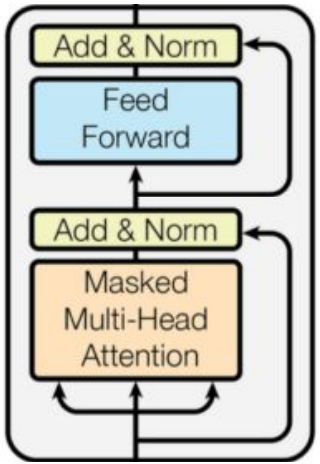
↳ Expensive to serve

Training data: 100s of billions of tokens or more

↳ Large-scale training

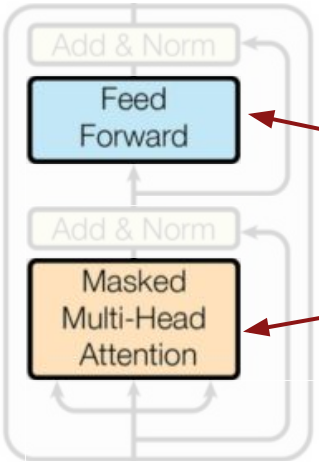
Question. How to make this work on real hardware?

Context



N×

Context



A lot of **matrix multiplications**!

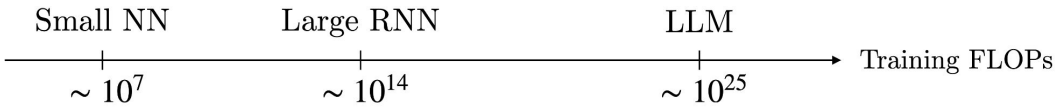
Answer. Leverage **GPUs** in a **smart** way!

Notations

📖 **FLOPs**

FL**oating-point **OP**erations**s

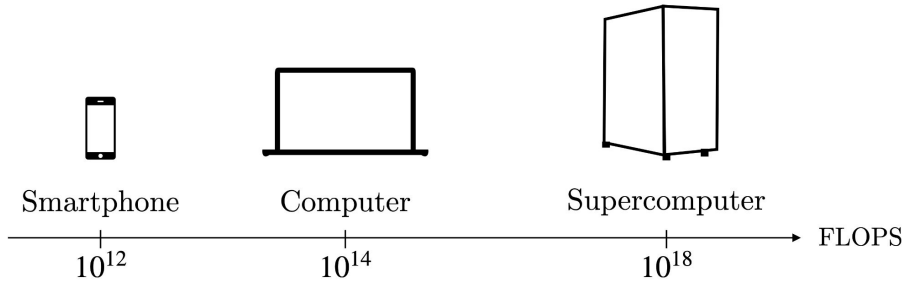
$f(\#\text{tokens}, \#\text{parameters})$



⚡ **FLOPS** or **FLOP/s**

***FL**oating-point **OP**erations per **S**econd*

Computational performance



Graphics Processing Unit (GPU)

Technical Specifications	H200 SXM[1]	H200 NVL[1]
FP64	34 TFLOPS	30 TFLOPS
FP64 Tensor Core	67 TFLOPS	60 TFLOPS
FP32	67 TFLOPS	60 TFLOPS
TF32 Tensor Core[2]	989 TFLOPS	835 TFLOPS
BFLOAT16 Tensor Core[2]	1,979 TFLOPS	1,671 TFLOPS
FP16 Tensor Core[2]	1,979 TFLOPS	1,671 TFLOPS
FP8 Tensor Core[2]	3,958 TFLOPS	3,341 TFLOPS
INT8 Tensor Core[2]	3,958 TFLOPS	3,341 TFLOPS
GPU Memory	141GB	141GB
GPU Memory Bandwidth	4.8TB/s	4.8TB/s
Decoders	7 NVDEC 7 JPEG	7 NVDEC 7 JPEG
Confidential Computing	Supported	Supported
Max Thermal Design Power (TDP)	Up to 700W (configurable)	Up to 600W (configurable)
Multi-Instance GPUs	Up to 7 MIGs @18GB each	Up to 7 MIGs @16.5GB each
Form Factor	SXM	PCIe Dual-slot air-cooled
Interconnect	NVIDIA NVLink: 900GB/s PCIe Gen5: 128GB/s	2- or 4-way NVIDIA NVLink bridge: 900GB/s per GPU PCIe Gen5: 128GB/s
Server Options	NVIDIA HGX™ H200 partner and NVIDIA-Certified Systems™ with 4 or 8 GPUs	NVIDIA MGX™ H200 NVL partner and NVIDIA-Certified Systems with up to 8 GPUs
NVIDIA AI Enterprise	Add-on	Included

Compute speed

Space to store intermediate computations

Transfer speed

Screenshot taken from NVIDIA H200 Tensor Core GPU, NVIDIA website.

Limitations

1 GPU **memory** is **limited**

Technical Specifications		
	H200 SXM[1]	H200 NVL[1]
FP64	34 TFLOPS	30 TFLOPS
FP64 Tensor Core	67 TFLOPS	60 TFLOPS
FP32	67 TFLOPS	60 TFLOPS
TF32 Tensor Core[2]	989 TFLOPS	835 TFLOPS
BFLOAT16 Tensor Core[2]	1,979 TFLOPS	1,671 TFLOPS
FP16 Tensor Core[2]	1,979 TFLOPS	1,671 TFLOPS
FP8 Tensor Core[2]	3,958 TFLOPS	3,341 TFLOPS
INT8 Tensor Core[2]	3,958 TFLOPS	3,341 TFLOPS
GPU Memory	141GB	141GB
GPU Memory Bandwidth	4.8TB/s	4.8TB/s
Decoders	7 NVDEC 7 JPEG	7 NVDEC 7 JPEG
Confidential Computing	Supported	Supported
Max Thermal Design Power (TDP)	Up to 700W (configurable)	Up to 600W (configurable)
Multi-Instance GPUs	Up to 7 MIGs @18GB each	Up to 7 MIGs @16.5GB each
Form Factor	SXM	PCIe Dual-slot air-cooled

Limited memory, **O(100 GB)**

Screenshot taken from NVIDIA H200 Tensor Core GPU, NVIDIA website.

Limitations

1. GPU **memory** is **limited**

Model size = **100 billion**

2 bytes of **space** per **parameter** (e.g. FP16)

Weight size = 200GB > Memory limit

For training, also need to hold **gradients**, **optimizer states** (on top of **activations**)

Limitations

1. GPU **memory** is **limited**

2. **Computationally** very **expensive**
   a. **Training**
   b. **Inference**

Limitations

1. GPU **memory** is **limited** → **How** do we make it **fit**?

2. **Computationally** very **expensive** → **How** do we make it **fast**?
   - a. Training
   - b. Inference

Questions throughout the lecture



How do we make it **fit**?



How do we make it **fast**?



Transformers & Large Language Models

Motivation

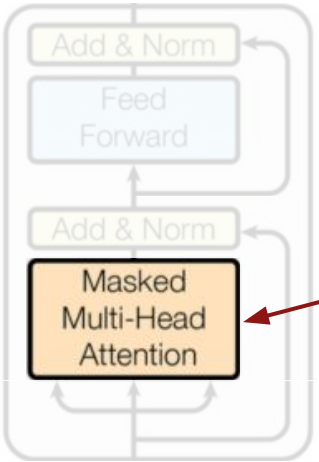
Flash attention

Distributed training

Inference optimizations

Discussion

Challenge with self-attention operation



$$\text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Huge matrix of size $n \times n$

Optimization via Flash Attention

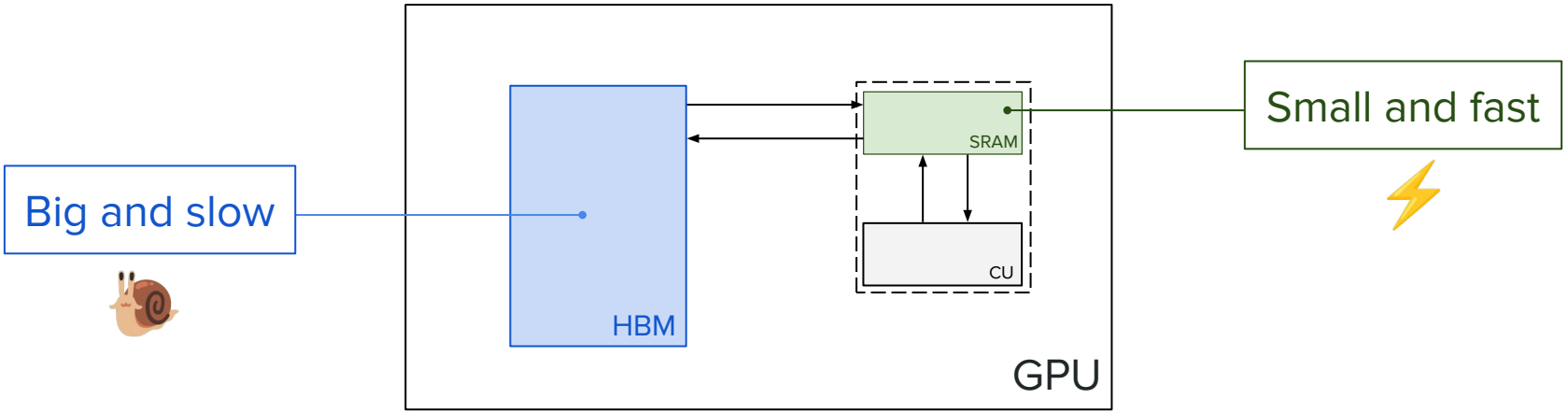
Strategy. Leverage components of GPU to speed up attention computations



FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness, Dao et al., 2022.

Optimization via Flash Attention

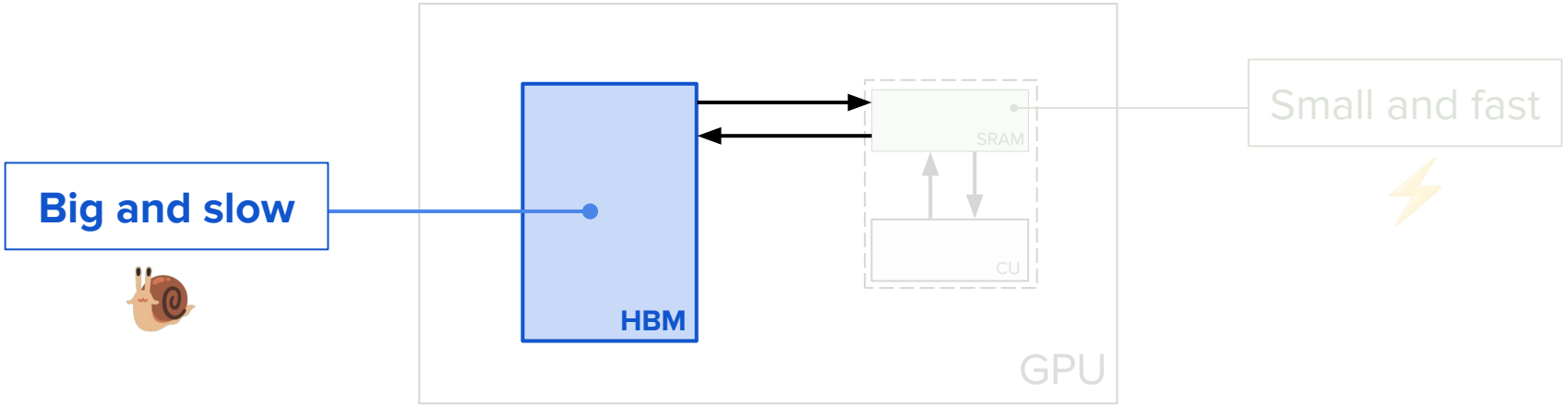
Strategy. Leverage components of GPU to speed up attention computations



FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness, Dao et al., 2022.

Optimization via Flash Attention

Idea. Reduce movement of data in/out of HBM



FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness, Dao et al., 2022.

Refresher on softmax

For $x = \begin{pmatrix} x_1 & x_2 & \cdots & x_n \end{pmatrix}$

We have $\text{softmax}(x)_i = \dfrac{e^{x_i}}{\boxed{\sum_{j=1}^{n} e^{x_j}}}$ ← **Dependency** on **full row** to be **processed**

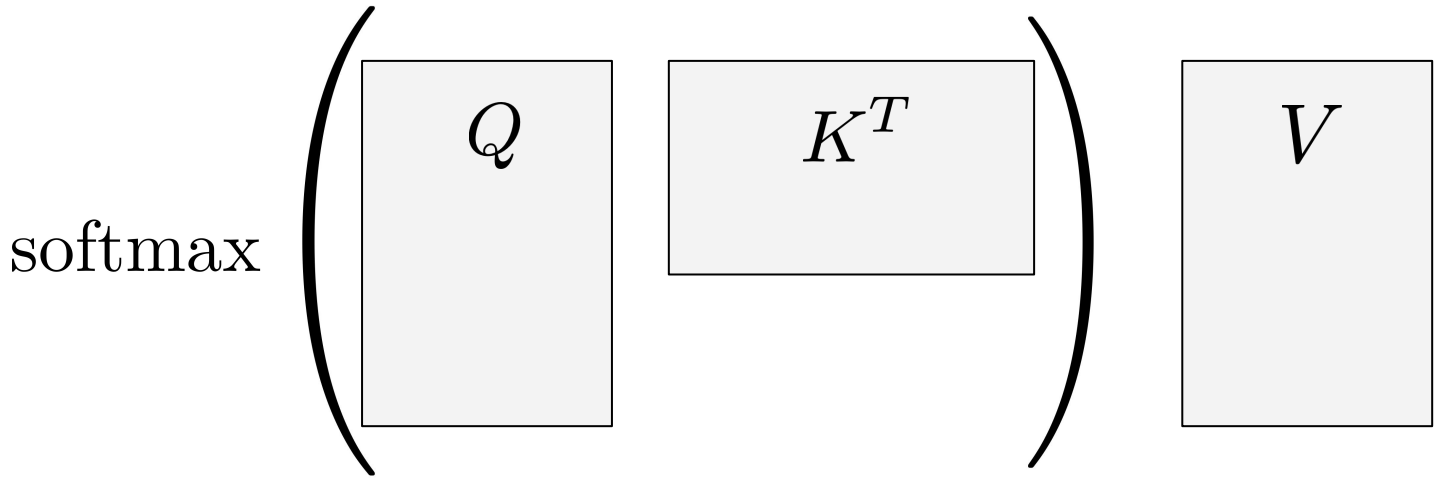
Standard self-attention computation

Context. "Standard" self-attention computation

$$\text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Standard self-attention computation

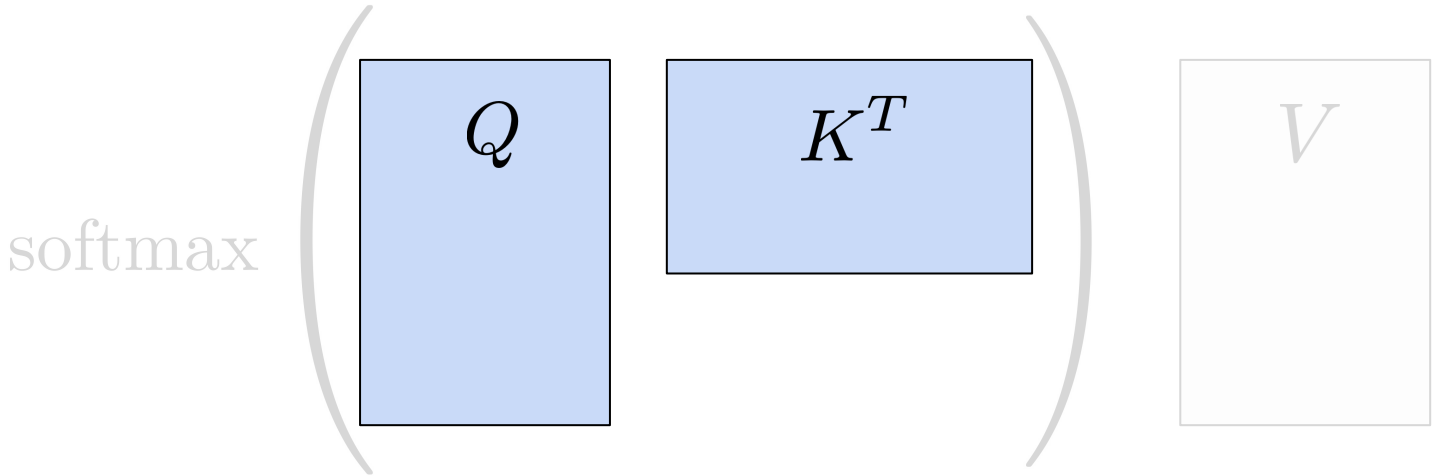
Context. "Standard" self-attention computation



FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness, Dao et al., 2022.

Standard self-attention computation

LOAD Q, K from **HBM** by blocks

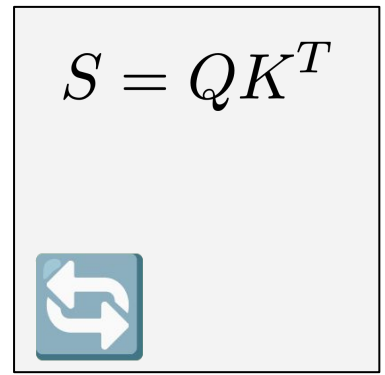


FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness, Dao et al., 2022.

Standard self-attention computation

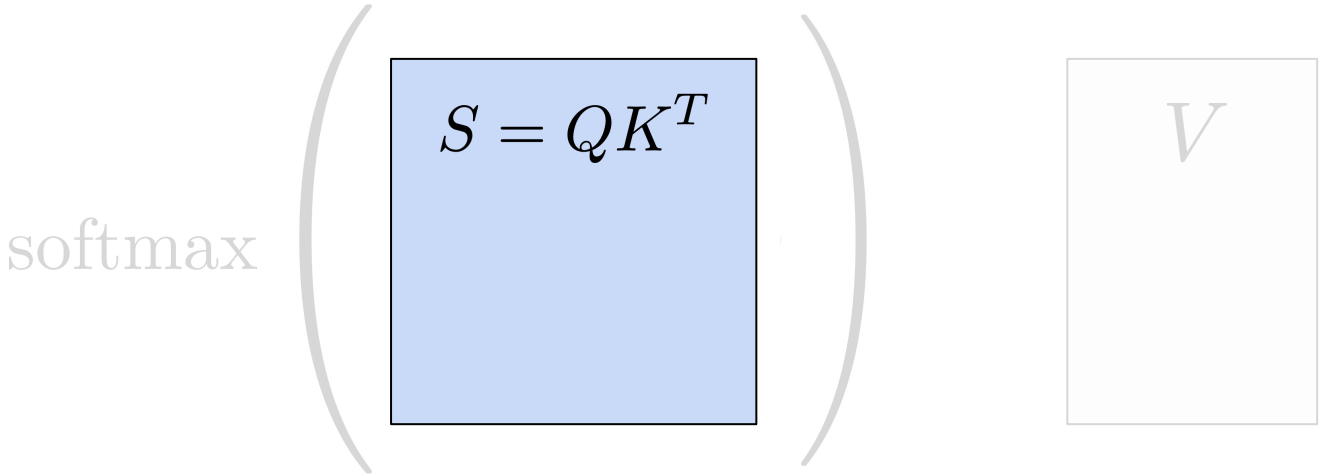
Compute S

$$\text{softmax}\left(S = QK^T \right) V$$



Standard self-attention computation

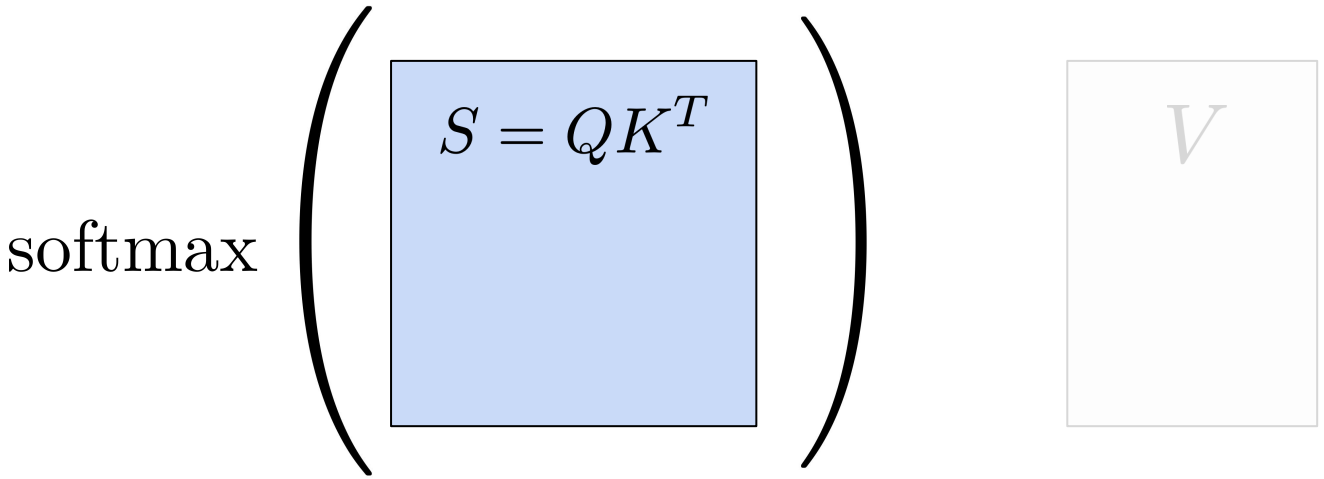
WRITE S to **HBM**



FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness, Dao et al., 2022.

Standard self-attention computation

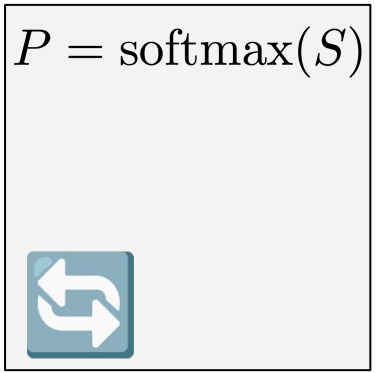
READ S from **HBM**



FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness, Dao et al., 2022.

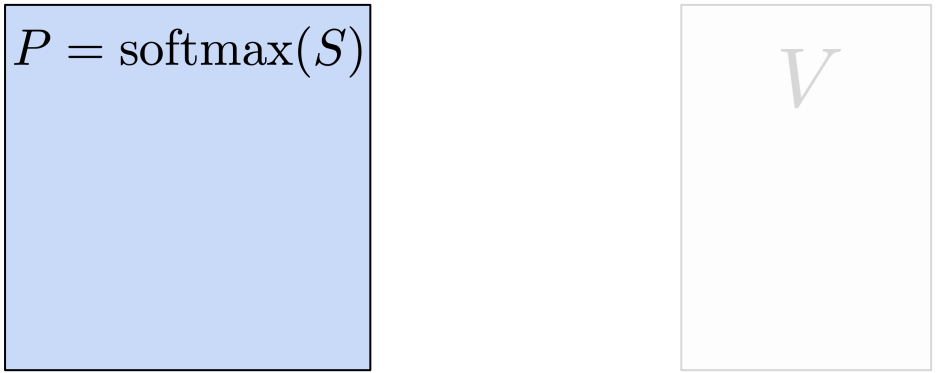
Standard self-attention computation

Compute P



Standard self-attention computation

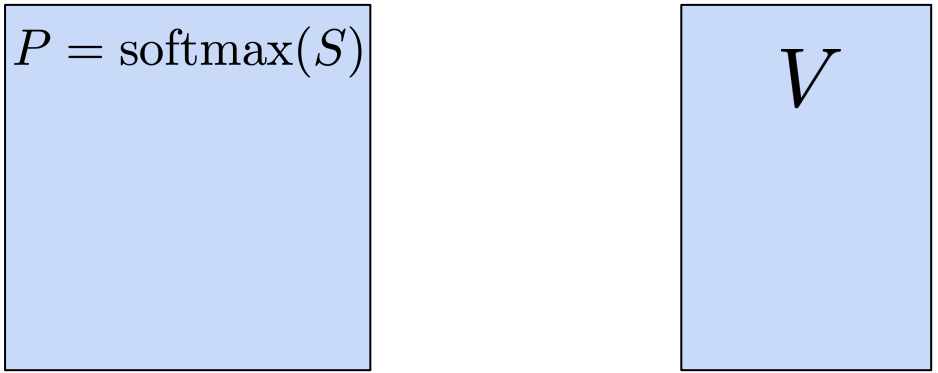
WRITE P to **HBM**



FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness, Dao et al., 2022.

Standard self-attention computation

LOAD P, V from **HBM** by blocks



FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness, Dao et al., 2022.

Standard self-attention computation

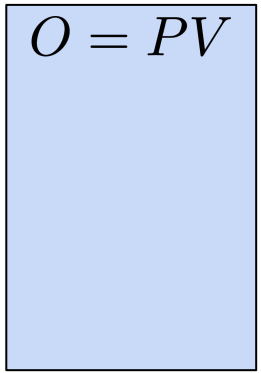
Compute O



FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness, Dao et al., 2022.

Standard self-attention computation

WRITE O to **HBM**



FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness, Dao et al., 2022.

Flash Attention trick on a simple example

Given $s = \begin{pmatrix} s_1 & s_2 & \cdots & s_n \end{pmatrix}$ and $v = \begin{pmatrix} v_1 \\ \vdots \\ v_n \end{pmatrix}$

Objective

We want to compute $\text{softmax}(s)\, v = \sum_{i=1}^{n} \frac{e^{s_i}}{\sum_{j=1}^{n} e^{s_j}} v_i$ in a **smart way**

Flash Attention trick on a simple example

$$\sum_{i=1}^{n} \frac{e^{s_i}}{\sum_{j=1}^{n} e^{s_j}} v_i$$

Flash Attention trick on a simple example

$$\sum_{i=1}^{n} \frac{e^{s_i}}{\sum_{j=1}^{n} e^{s_j}} v_i = \frac{\boxed{\sum_{i=1}^{n} e^{s_i} v_i}}{\boxed{\sum_{j=1}^{n} e^{s_j}}}$$

Numerator N

Denominator D

Trick. Compute numerator and denominator incrementally

Flash Attention trick on a simple example

$$\sum_{i=1}^{n} \frac{e^{s_i}}{\sum_{j=1}^{n} e^{s_j}} v_i = \frac{N}{D}$$

Computed iteratively

Trick. Compute numerator and denominator incrementally

Flash Attention trick on a simple example

$$\sum_{i=1}^{n} \frac{e^{s_i}}{\sum_{j=1}^{n} e^{s_j}} v_i = \frac{N_n}{D_n}$$

$$N_t = \sum_{i=1}^{t} e^{s_i} v_i$$

$$D_t = \sum_{j=1}^{t} e^{s_j}$$

Trick. Compute numerator and denominator incrementally

Flash Attention trick on a simple example

$$\text{softmax}\left(\begin{pmatrix} \boxed{s_1} & s_2 & s_3 \end{pmatrix}\right) \begin{pmatrix} \boxed{v_1} \\ v_2 \\ v_3 \end{pmatrix}$$

$$N_1 = e^{s_1} v_1$$

$$D_1 = e^{s_1}$$

Flash Attention trick on a simple example

$$\text{softmax}\left(\begin{pmatrix} s_1 & \boxed{s_2} & s_3 \end{pmatrix}\right) \begin{pmatrix} v_1 \\ \boxed{v_2} \\ v_3 \end{pmatrix}$$

$$N_2 = e^{s_1} v_1 + e^{s_2} v_2$$

$$D_2 = e^{s_1} + e^{s_2}$$

Flash Attention trick on a simple example

$$\text{softmax}\left(\begin{pmatrix} s_1 & s_2 & \boxed{s_3} \end{pmatrix}\right) \begin{pmatrix} v_1 \\ v_2 \\ \boxed{v_3} \end{pmatrix}$$

$$N_3 = e^{s_1} v_1 + e^{s_2} v_2 + e^{s_3} v_3$$

$$D_3 = e^{s_1} + e^{s_2} + e^{s_3}$$

Flash Attention trick on a simple example

$$\text{softmax}\left(\begin{pmatrix} s_1 & s_2 & s_3 \end{pmatrix}\right) \begin{pmatrix} v_1 \\ v_2 \\ v_3 \end{pmatrix}$$

$$N_{t+1} = N_t + e^{s_{t+1}} v_{t+1}$$

$$D_{t+1} = D_t + e^{s_{t+1}}$$

Flash Attention trick on a simple example

$$\underbrace{\text{softmax}\left(\begin{pmatrix} s_1 & s_2 & s_3 \end{pmatrix}\right)\begin{pmatrix} v_1 \\ v_2 \\ v_3 \end{pmatrix}}_{} = \frac{N_3}{D_3}$$

Flash Attention trick on a simple example

⚠️ e^{s_i} can be big and **cause numerical instabilities** ⚠️

We note that: $$\frac{N}{D} = \frac{\boxed{C} \times N}{\boxed{C} \times D}$$

Idea. Choose this constant wisely! $C = e^{-\max\limits_{1 \leqslant i \leqslant n} s_i}$

Flash Attention trick on a simple example

$$\text{softmax}\left(\begin{pmatrix} \boxed{s_1} & s_2 & s_3 \end{pmatrix}\right) \begin{pmatrix} \boxed{v_1} \\ v_2 \\ v_3 \end{pmatrix}$$

$$m_1 = s_1$$

$$N_1 = e^{s_1} v_1$$

$$D_1 = e^{s_1}$$

Flash Attention trick on a simple example

$$\text{softmax}\left(\begin{pmatrix} s_1 & s_2 & s_3 \end{pmatrix}\right) \begin{pmatrix} v_1 \\ v_2 \\ v_3 \end{pmatrix}$$

$$m_1 = s_1$$

$$N_1 = e^{s_1} v_1$$

$$D_1 = e^{s_1}$$

Flash Attention trick on a simple example

$$\text{softmax}\left(\begin{pmatrix} s_1 & s_2 & s_3 \end{pmatrix}\right) \begin{pmatrix} v_1 \\ v_2 \\ v_3 \end{pmatrix}$$

$$m_1 = s_1$$

$$N_1 = e^{-m_1} \times e^{s_1} v_1$$

$$D_1 = e^{-m_1} \times e^{s_1}$$

Flash Attention trick on a simple example

$$\text{softmax}\left(\begin{pmatrix} s_1 & s_2 & s_3 \end{pmatrix}\right) \begin{pmatrix} v_1 \\ v_2 \\ v_3 \end{pmatrix}$$

$$m_1 = s_1$$

$$N_1 = e^{s_1 - m_1} v_1$$

$$D_1 = e^{s_1 - m_1}$$

Flash Attention trick on a simple example

$$\text{softmax}\left(\begin{pmatrix} s_1 & \boxed{s_2} & s_3 \end{pmatrix}\right) \begin{pmatrix} v_1 \\ \boxed{v_2} \\ v_3 \end{pmatrix}$$

$$m_2 = \max(s_1, s_2)$$

$$N_2 = e^{m_1 - m_2} \times e^{s_1 - m_1} v_1 + e^{s_2 - m_2} v_2$$

$$D_2 = e^{m_1 - m_2} \times e^{s_1 - m_1} + e^{s_2 - m_2}$$

Flash Attention trick on a simple example

$$\text{softmax}\left(\begin{pmatrix} s_1 & s_2 & s_3 \end{pmatrix}\right) \begin{pmatrix} v_1 \\ v_2 \\ v_3 \end{pmatrix}$$

$$m_{t+1} = \max(m_t, s_{t+1}) \qquad \alpha_t = e^{m_t - m_{t+1}}$$

New scaling factor

Rescale running sum computed so far

Flash Attention trick on a simple example

$$\text{softmax}\left(\begin{pmatrix} s_1 & s_2 & s_3 \end{pmatrix}\right) \begin{pmatrix} v_1 \\ v_2 \\ v_3 \end{pmatrix}$$

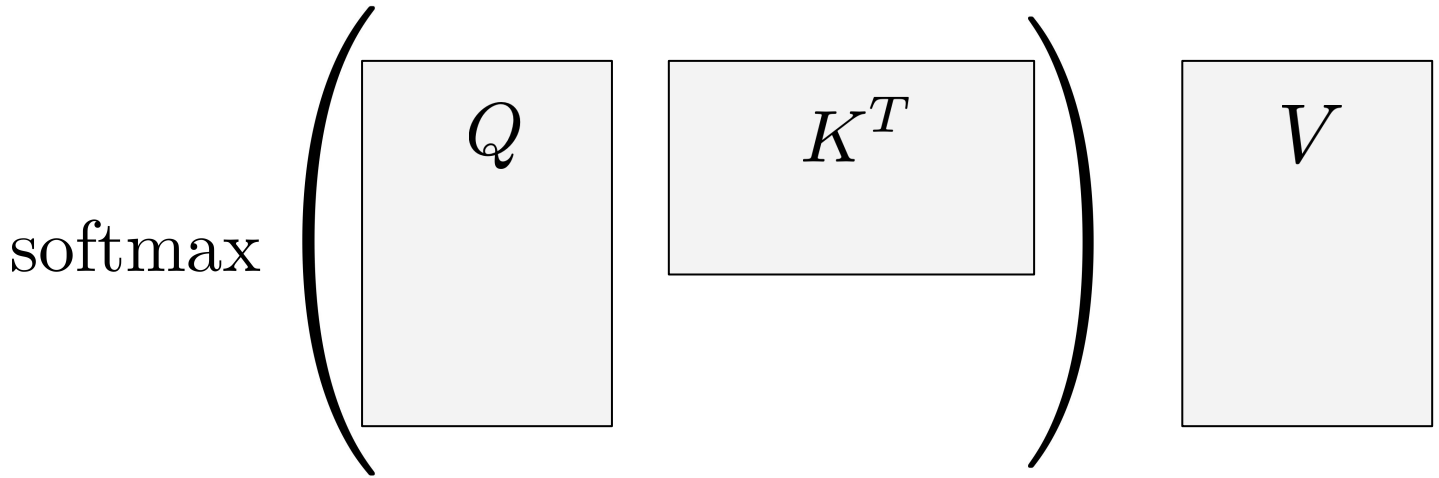
$$m_{t+1} = \max(m_t, s_{t+1}) \qquad \alpha_t = e^{m_t - m_{t+1}}$$

$$N_{t+1} = \alpha_t N_t + e^{s_{t+1} - m_{t+1}} v_{t+1}$$

$$D_{t+1} = \alpha_t D_t + e^{s_{t+1} - m_{t+1}}$$

Flash Attention in action

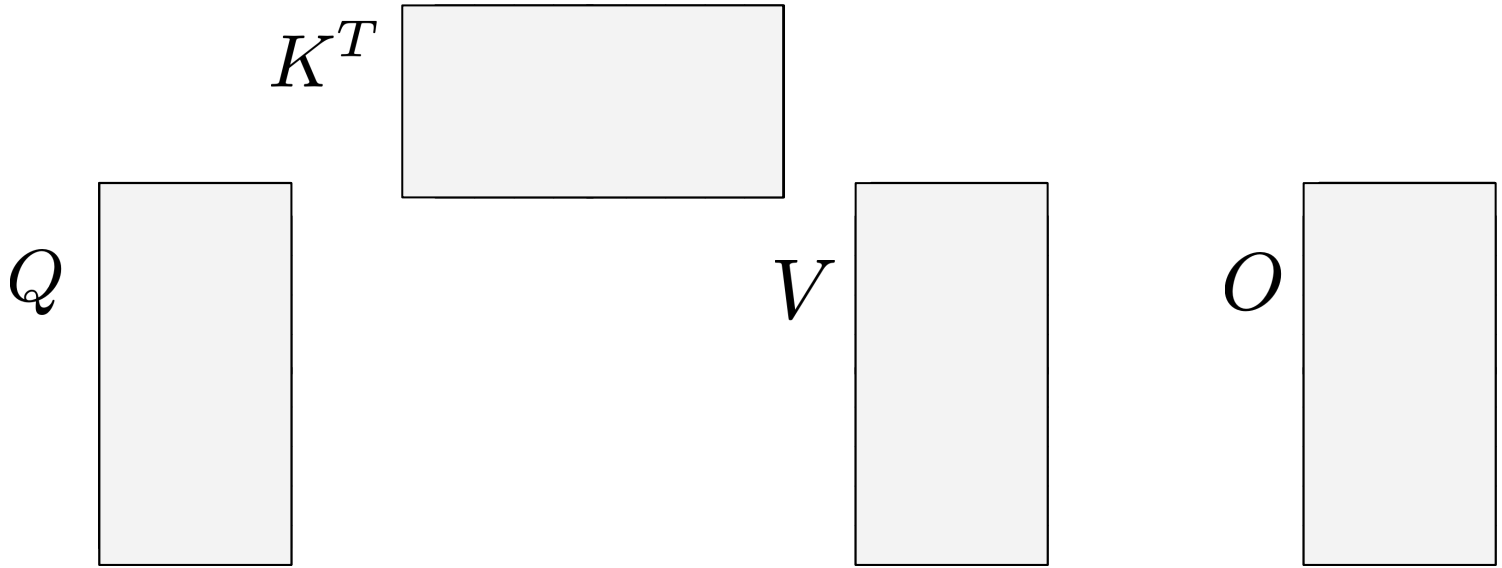
1st idea. Minimize reads/writes to **HBM** with clever "tiling" via **SRAM**



FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness, Dao et al., 2022.

Flash Attention in action

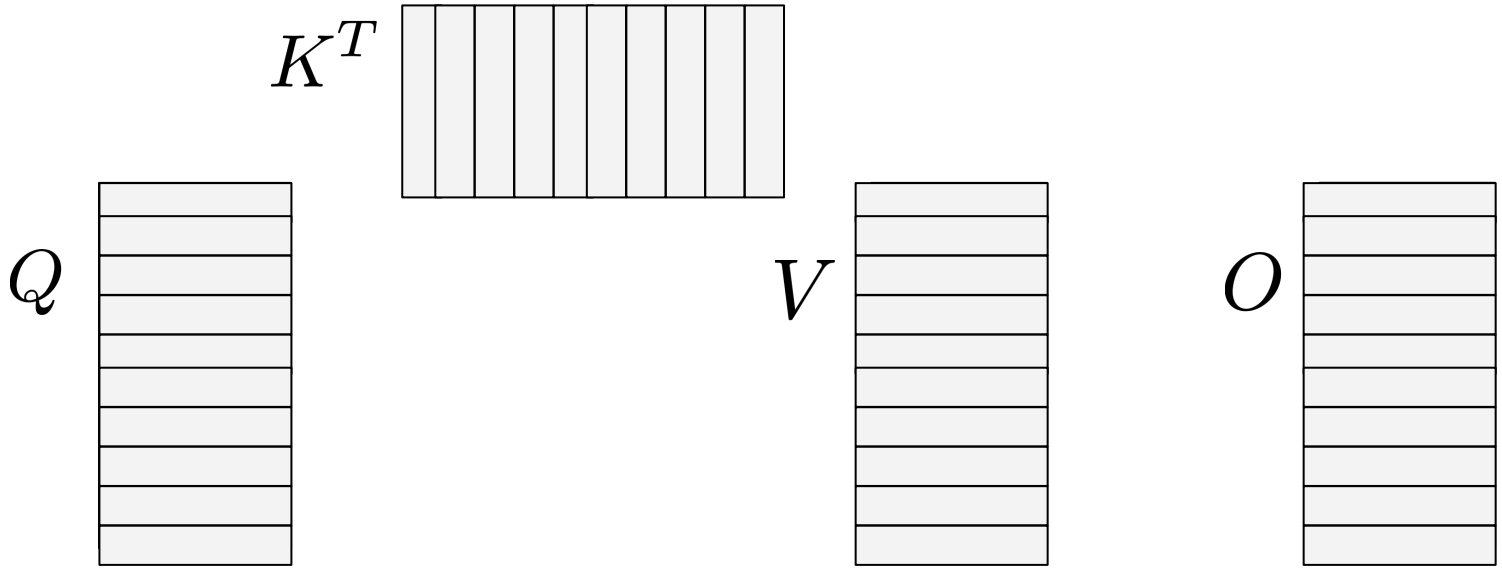
1st idea. Minimize reads/writes to **HBM** with clever "tiling" via **SRAM**



FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness, Dao et al., 2022.

Flash Attention in action

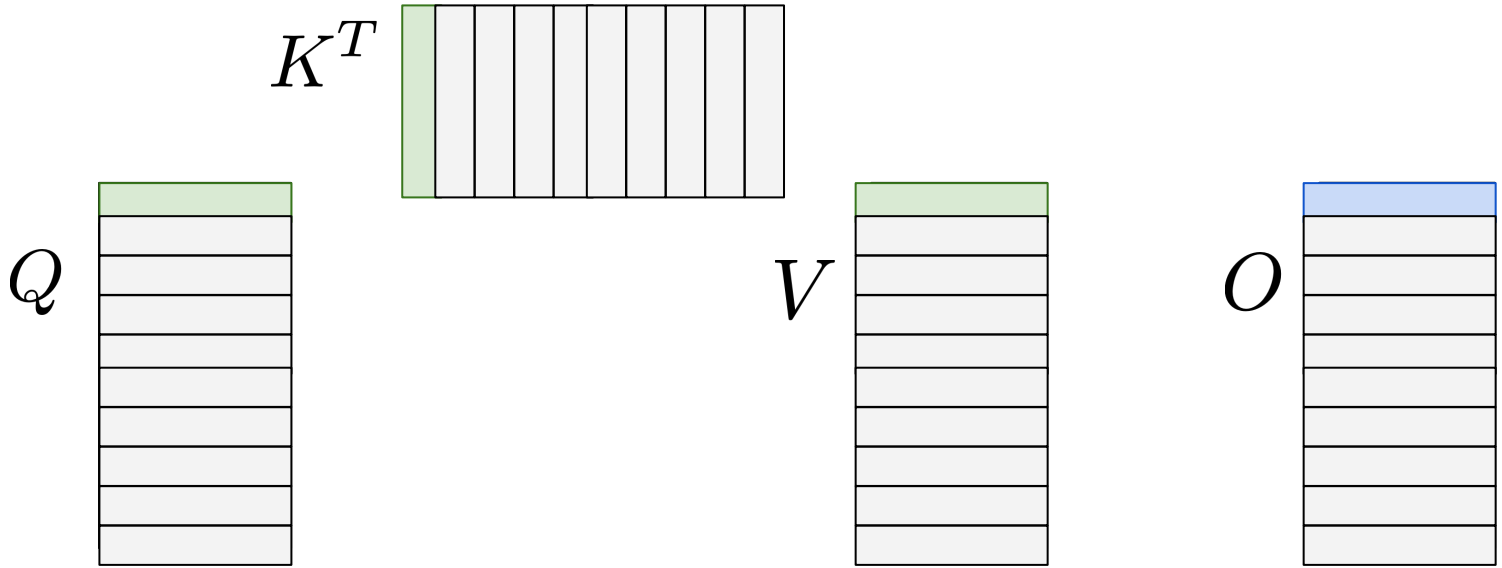
1st idea. Minimize reads/writes to **HBM** with clever "tiling" via **SRAM**



FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness, Dao et al., 2022.

Flash Attention in action

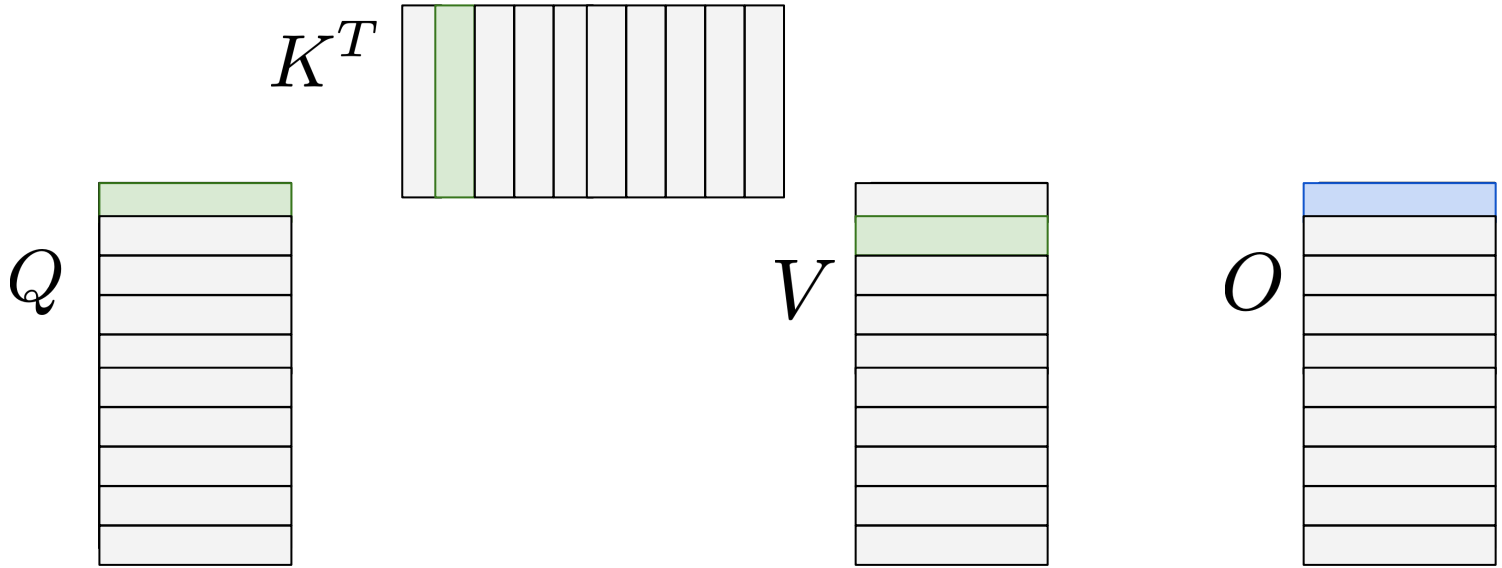
Load blocks of Q, K, V to **SRAM**, compute block of O and write the result to **HBM**



FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness, Dao et al., 2022.

Flash Attention in action

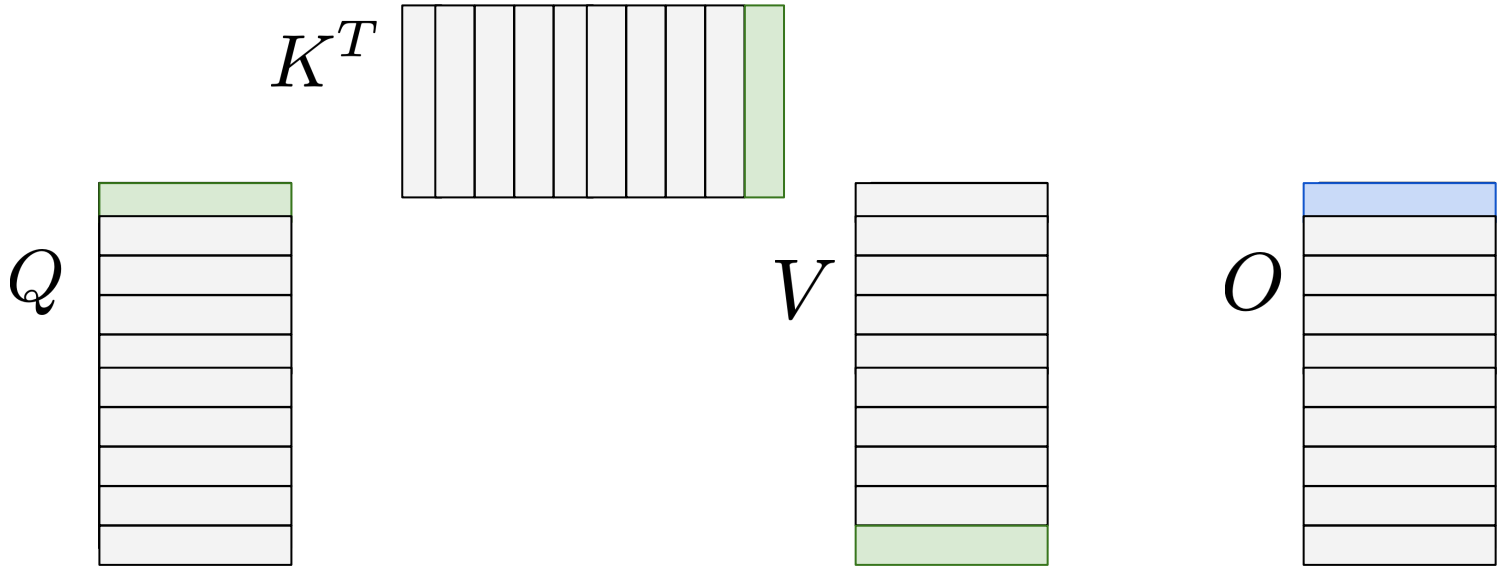
Load blocks of Q, K, V to **SRAM**, compute block of O and write the result to **HBM**



FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness, Dao et al., 2022.

Flash Attention in action

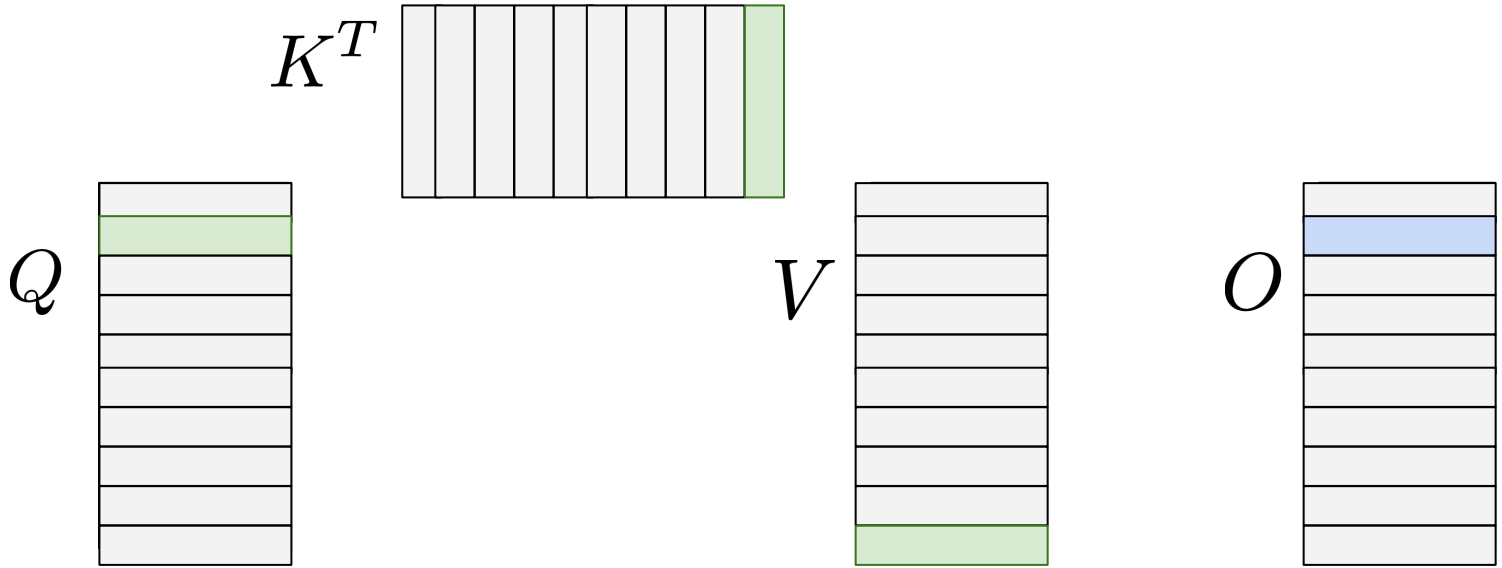
Load blocks of Q, K, V to **SRAM**, compute block of O and write the result to **HBM**



FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness, Dao et al., 2022.

Flash Attention in action

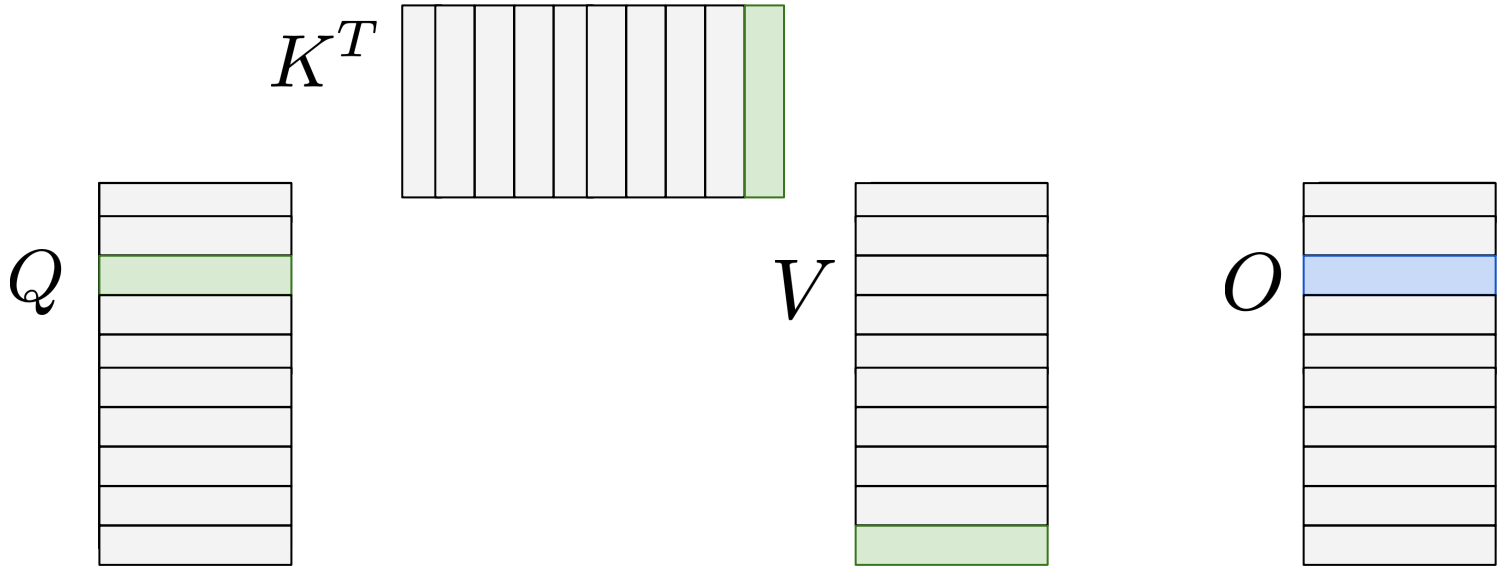
Load blocks of Q, K, V to **SRAM**, compute block of O and write the result to **HBM**



FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness, Dao et al., 2022.

Flash Attention in action

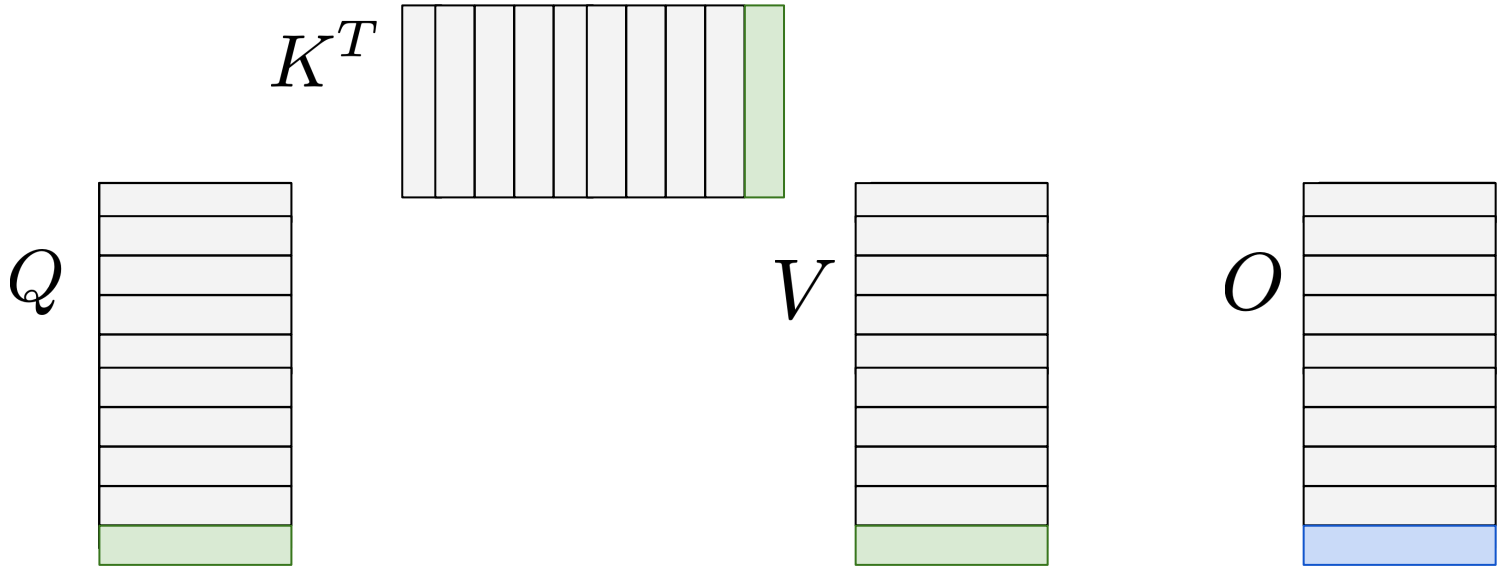
Load blocks of Q, K, V to **SRAM**, compute block of O and write the result to **HBM**



FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness, Dao et al., 2022.

Flash Attention in action

Load blocks of Q, K, V to **SRAM**, compute block of O and write the result to **HBM**



FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness, Dao et al., 2022.

Flash Attention in action

Far fewer reads

and writes to **HBM**!

Flash Attention during training

2nd idea. Sometimes, it is better to **recompute** instead of storing

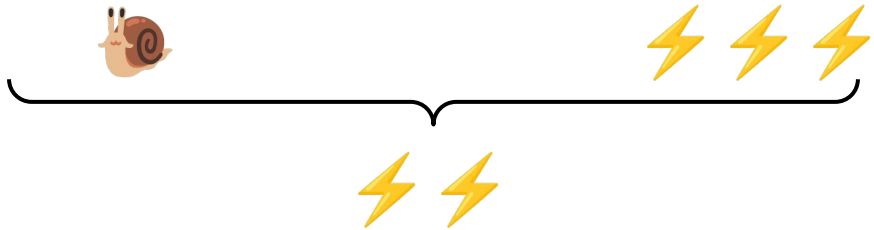
Before	After
Store S, P in the **HBM**	Recompute S, P with tiling via **SRAM**

FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness, Dao et al., 2022.

Flash Attention during training

2nd idea. Sometimes, it is better to **recompute** instead of storing

Before	After
Store S, P in the **HBM**	Recompute S, P with tiling via **SRAM**



FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness, Dao et al., 2022.

Discussion

Results. Flash Attention leads to a significant speedup with **exact** computation

Attention	Standard	FlashAttention
GFLOPs	66.6	75.2
HBM R/W (GB)	40.3	4.4
Runtime (ms)	41.7	7.3

More FLOPs, but **less runtime**!!

Variants. Optimizations for newer GPU versions

- Flash attention 2
- Flash attention 3 (Hopper)
- Flash attention 4 (Blackwell)

FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness, Dao et al., 2022.

Summary for Flash Attention



How do we make it **fit**?

↳ **Avoid** materializing the $n \times n$ **attention** matrix

Summary for Flash Attention

How do we make it **fit**?

↳ **Avoid** materializing the $n \times n$ **attention** matrix



How do we make it **fast**?

↳ **Minimize** reads/writes to **HBM**



Transformers & Large Language Models

Motivation

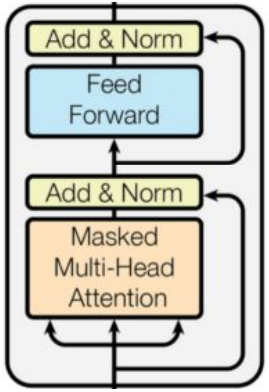
Flash attention

Distributed training

Inference optimizations

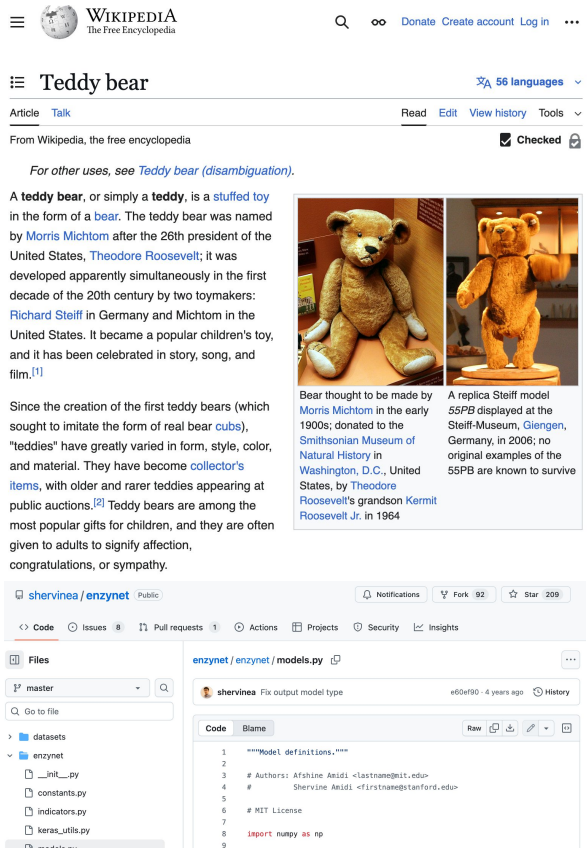
Discussion

Setup of LLM training



Nx

+



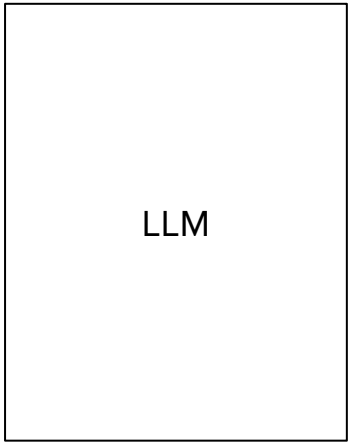
+

(Many) GPU(s)!

Attention Is All You Need, Vaswani et al., 2017.

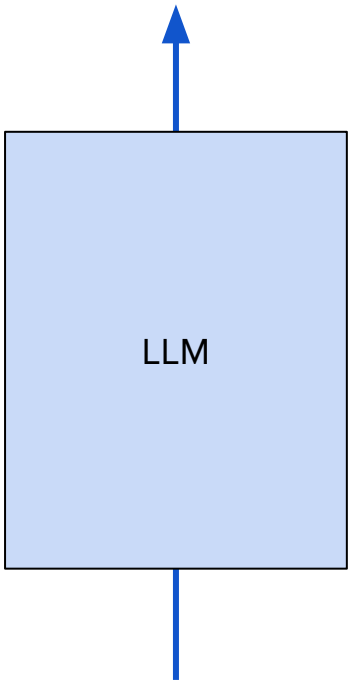
Setup of LLM training

Initialization



Model parameters. O(billions) - O(trillions)

Setup of LLM training



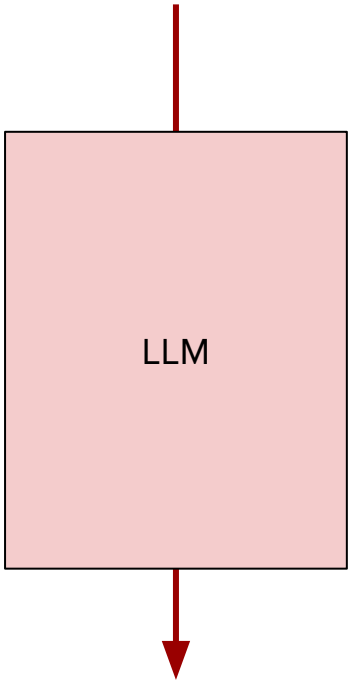
Forward pass. Compute loss.

Activations. Needed to compute the loss and gradients

$$\mathscr{L}(\theta_t)$$

Function of model size, batch size, context length

Setup of LLM training

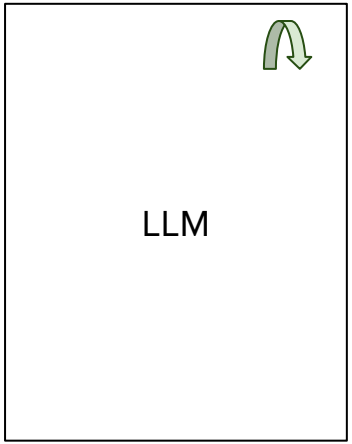


Backward pass. Compute gradients.

Gradients. Needed for weights update

$$\nabla \mathscr{L}(\theta_t)$$

Setup of LLM training



Weights update. Update model parameters.

Optimizer state. Updates the weights

$$\theta_{t+1} \longleftarrow \theta_t - \alpha \frac{m_t}{\sqrt{v_t} + \epsilon}$$

Adam optimizer

with: $m_t \leftarrow \beta_1 m_{t-1} + (1-\beta_1)\nabla \mathscr{L}(\theta_t)$

$v_t \leftarrow \beta_2 v_{t-1} + (1-\beta_2)(\nabla \mathscr{L}(\theta_t))^2$

Super Study Guide: Transformers and Large Language Models, Amidi et al., 2024.
Suggested readings: *Decoupled Weight Decay Regularization*, Loshchilov et al., 2017 ; *Adam: A Method for Stochastic Optimization*, Kingma et al., 2014.

Bottleneck: memory

Technical Specifications		
	H200 SXM[1]	**H200 NVL**[1]
FP64	34 TFLOPS	30 TFLOPS
FP64 Tensor Core	67 TFLOPS	60 TFLOPS
FP32	67 TFLOPS	60 TFLOPS
TF32 Tensor Core[2]	989 TFLOPS	835 TFLOPS
BFLOAT16 Tensor Core[2]	1,979 TFLOPS	1,671 TFLOPS
FP16 Tensor Core[2]	1,979 TFLOPS	1,671 TFLOPS
FP8 Tensor Core[2]	3,958 TFLOPS	3,341 TFLOPS
INT8 Tensor Core[2]	3,958 TFLOPS	3,341 TFLOPS
GPU Memory	141GB	141GB
GPU Memory Bandwidth	4.8TB/s	4.8TB/s
Decoders	7 NVDEC 7 JPEG	7 NVDEC 7 JPEG
Confidential Computing	Supported	Supported
Max Thermal Design Power (TDP)	Up to 700W (configurable)	Up to 600W (configurable)
Multi-Instance GPUs	Up to 7 MIGs @18GB each	Up to 7 MIGs @16.5GB each
Form Factor	SXM	PCIe Dual-slot air-cooled
Interconnect	NVIDIA NVLink: 900GB/s PCIe Gen5: 128GB/s	2- or 4-way NVIDIA NVLink bridge: 900GB/s per GPU PCIe Gen5: 128GB/s
Server Options	NVIDIA HGX™ H200 partner and NVIDIA-Certified Systems™ with 4 or 8 GPUs	NVIDIA MGX™ H200 NVL partner and NVIDIA-Certified Systems with up to 8 GPUs
NVIDIA AI Enterprise	Add-on	Included

Screenshot taken from NVIDIA H200 Tensor Core GPU, NVIDIA website.

Bottleneck: memory

Technical Specifications		
	H200 SXM[1]	H200 NVL[1]
FP64	34 TFLOPS	30 TFLOPS
FP64 Tensor Core	67 TFLOPS	60 TFLOPS
FP32	67 TFLOPS	60 TFLOPS
TF32 Tensor Core[2]	989 TFLOPS	835 TFLOPS
BFLOAT16 Tensor Core[2]	1,979 TFLOPS	1,671 TFLOPS
FP16 Tensor Core[2]	1,979 TFLOPS	1,671 TFLOPS
FP8 Tensor Core[2]	3,958 TFLOPS	3,341 TFLOPS
INT8 Tensor Core[2]	3,958 TFLOPS	3,341 TFLOPS
GPU Memory	141GB	141GB
GPU Memory Bandwidth	4.8TB/s	4.8TB/s
Decoders	7 NVDEC 7 JPEG	7 NVDEC 7 JPEG
Confidential Computing	Supported	Supported
Max Thermal Design Power (TDP)	Up to 700W (configurable)	Up to 600W (configurable)
Multi-Instance GPUs	Up to 7 MIGs @18GB each	Up to 7 MIGs @16.5GB each
Form Factor	SXM	PCIe Dual-slot air-cooled
Interconnect	NVIDIA NVLink: 900GB/s PCIe Gen5: 128GB/s	2- or 4-way NVIDIA NVLink bridge: 900GB/s per GPU PCIe Gen5: 128GB/s
Server Options	NVIDIA HGX™ H200 partner and NVIDIA-Certified Systems™ with 4 or 8 GPUs	NVIDIA MGX™ H200 NVL partner and NVIDIA-Certified Systems with up to 8 GPUs
NVIDIA AI Enterprise	Add-on	Included

Limited memory, O(100 GB)

Screenshot taken from NVIDIA H200 Tensor Core GPU, NVIDIA website.

Data parallelism

DP = **D**ata **P**arallelism

Idea. Divide batch of data across devices



parameters

gradients

optimizer state

Data parallelism with ZeRO

Objective. Reduce redundant information across devices

ZeRO = **Ze**ro **R**edundancy **O**ptimizer

Variations. ZeRO-1: Shard **optimizer state**



ZeRO: Memory Optimizations Toward Training Trillion Parameter Models, *Rajbhandari et al., 2019.*

Data parallelism with ZeRO

Objective. Reduce redundant information across devices

ZeRO = **Ze**ro **R**edundancy **O**ptimizer

Variations. ZeRO-2: Shard **optimizer state** + **gradients**



ZeRO: Memory Optimizations Toward Training Trillion Parameter Models, *Rajbhandari et al., 2019.*

Data parallelism with ZeRO

Objective. Reduce redundant information across devices

ZeRO = **Ze**ro **R**edundancy **O**ptimizer

Variations. ZeRO-3: Shard **optimizer state** + **gradients** + **parameters**



ZeRO: Memory Optimizations Toward Training Trillion Parameter Models, *Rajbhandari et al., 2019.*

Model parallelism

Idea. Split the model computations across several devices

- **T**ensor **P**arallelism **(TP)**

Self-attention layer

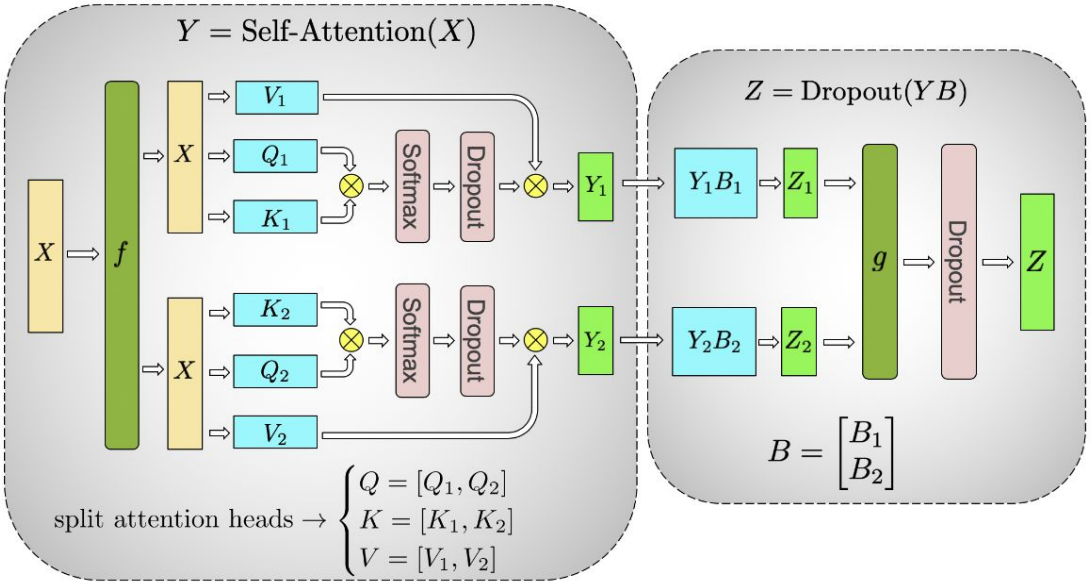


Figure from Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism, Shoeybi et al., 2019.

Model parallelism

Idea. Split the model computations across several devices

- **T**ensor **P**arallelism **(TP)**

Feedforward neural network layer

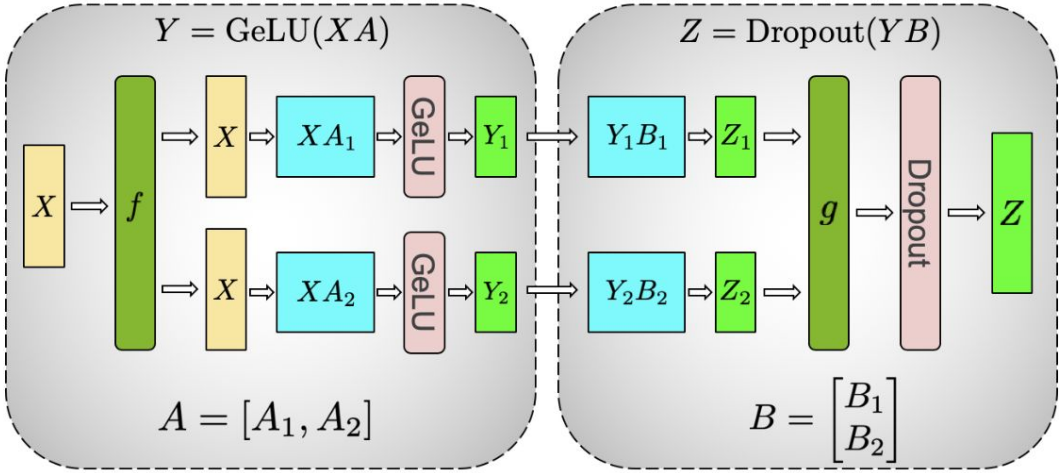


Figure from Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism, Shoeybi et al., 2019.

Model parallelism

Idea. Split the model computations across several devices

- Tensor Parallelism **(TP)**
- **P**ipeline **P**arallelism **(PP)**

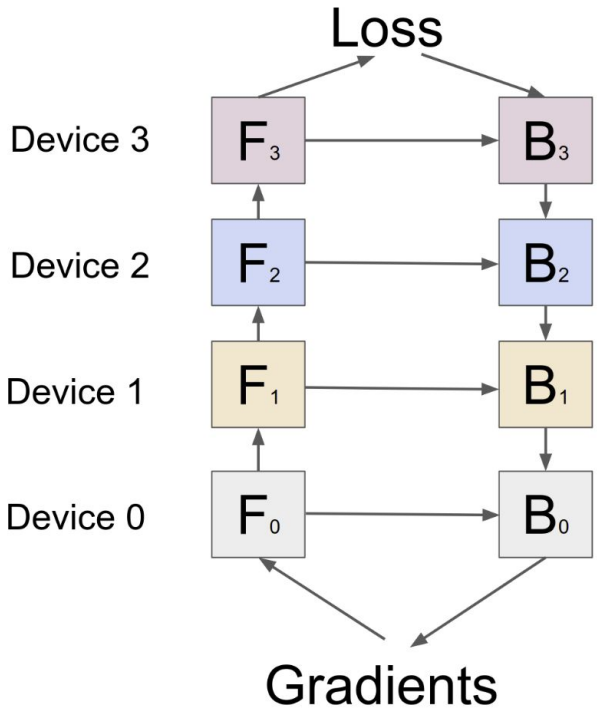
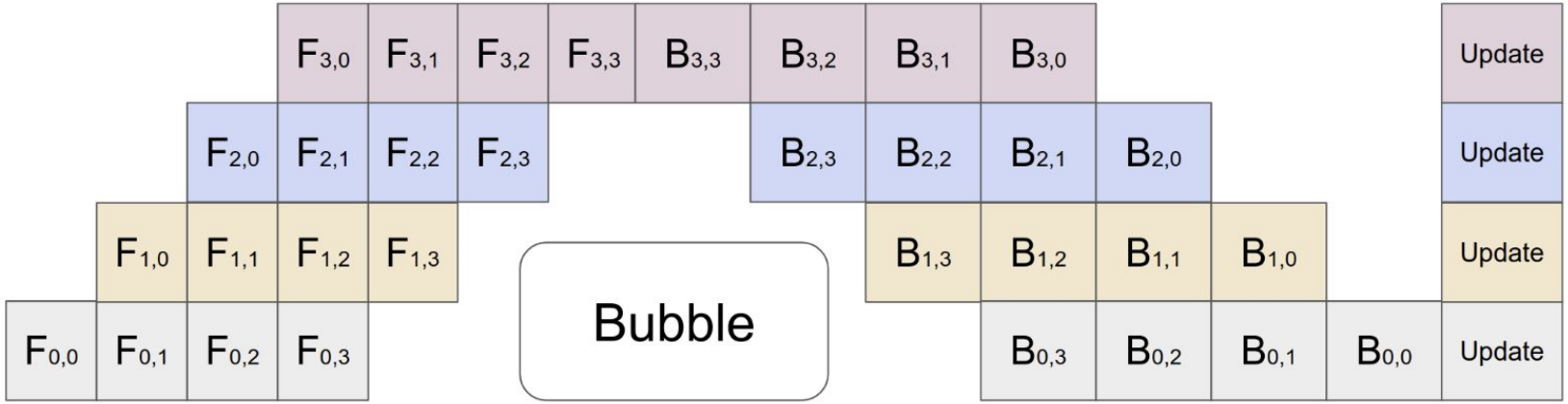


Figure from GPipe: Efficient Training of Giant Neural Networks using Pipeline Parallelism, Huang et al., 2018.

Model parallelism

Idea. Split the model computations across several devices

- Tensor Parallelism **(TP)**
- **P**ipeline **P**arallelism **(PP)**



GPipe: Efficient Training of Giant Neural Networks using Pipeline Parallelism, Huang et al., 2018.

Model parallelism

Idea. Split the model computations across several devices

- Tensor Parallelism **(TP)**
- Pipeline Parallelism **(PP)**
- **C**ontext **P**arallelism **(CP)**

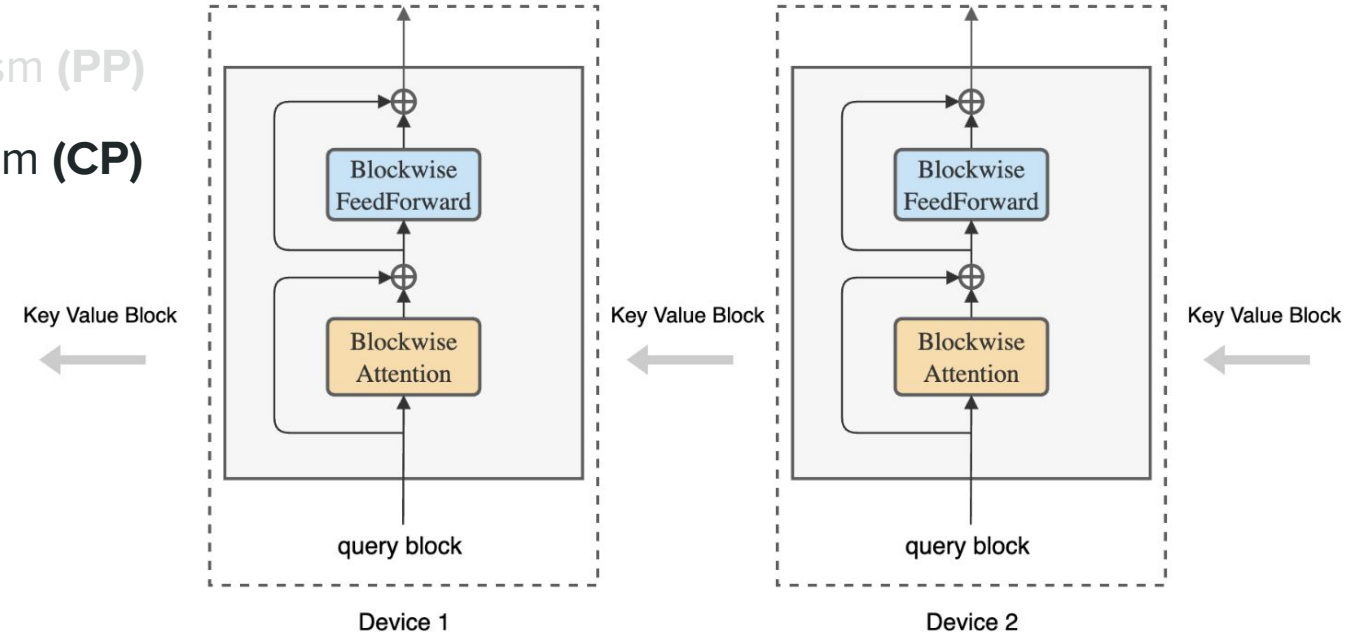


Figure from Ring Attention with Blockwise Transformers for Near-Infinite Context, Liu et al., 2023.

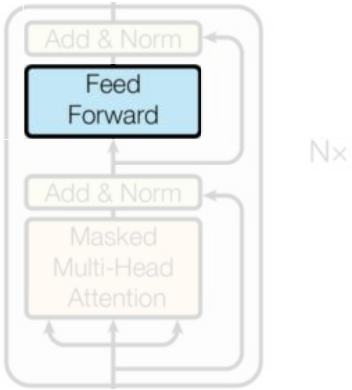
Model parallelism

Idea. Split the model computations across several devices

- **T**ensor **P**arallelism **(TP)**
- **P**ipeline **P**arallelism **(PP)**
- **C**ontext **P**arallelism **(CP)**
- **E**xpert **P**arallelism **(EP)**



MoE in Transformer-based models



Routing done **for each token**!

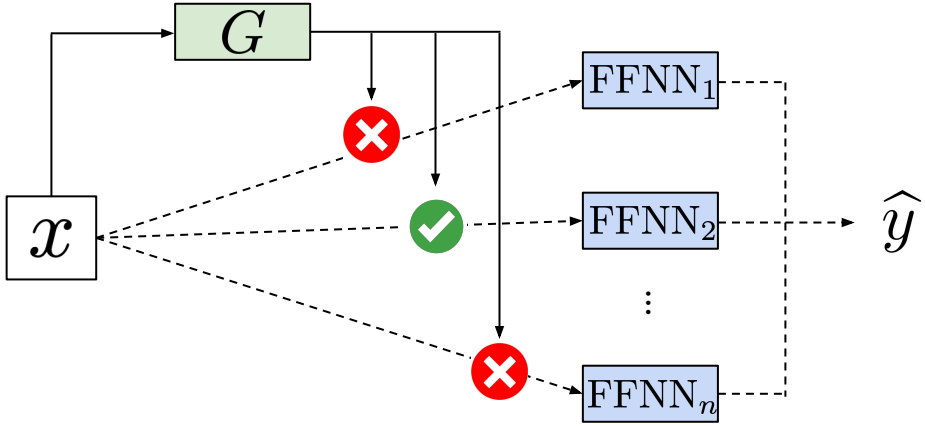


Figure adapted from Attention Is All You Need, Vaswani et al., 2017.

Model parallelism

Idea. Split the model computations across several devices

- Tensor Parallelism **(TP)**
- Pipeline Parallelism **(PP)**
- Context Parallelism **(CP)**
- **E**xpert **P**arallelism **(EP)**

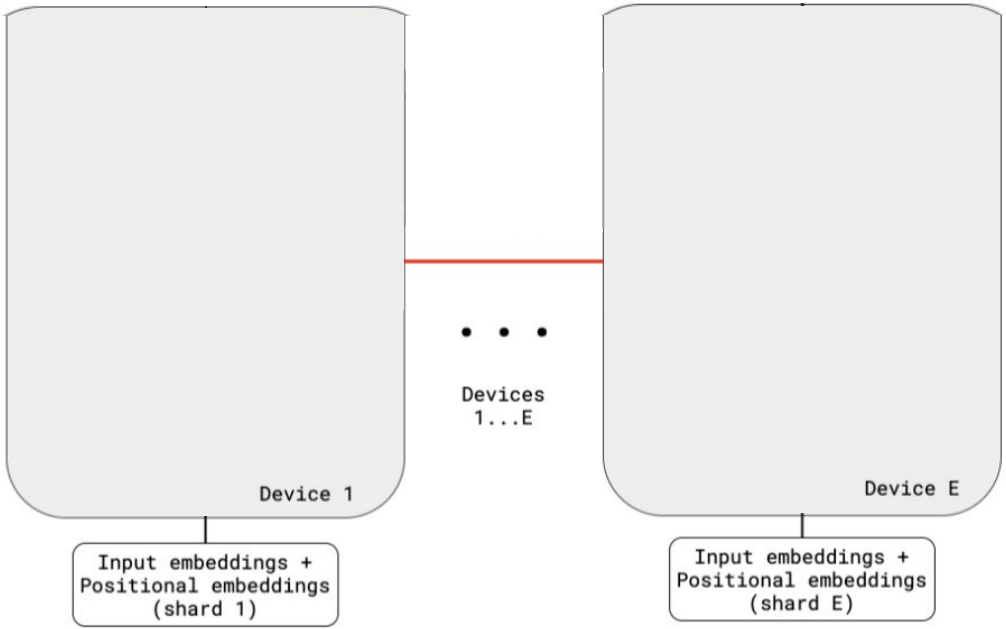


Figure adapted from GShard: Scaling Giant Models with Conditional Computation and Automatic Sharding, Lepikhin et al., 2020.

Model parallelism

Idea. Split the model computations across several devices

- Tensor Parallelism **(TP)**
- Pipeline Parallelism **(PP)**
- Context Parallelism **(CP)**
- **E**xpert **P**arallelism **(EP)**

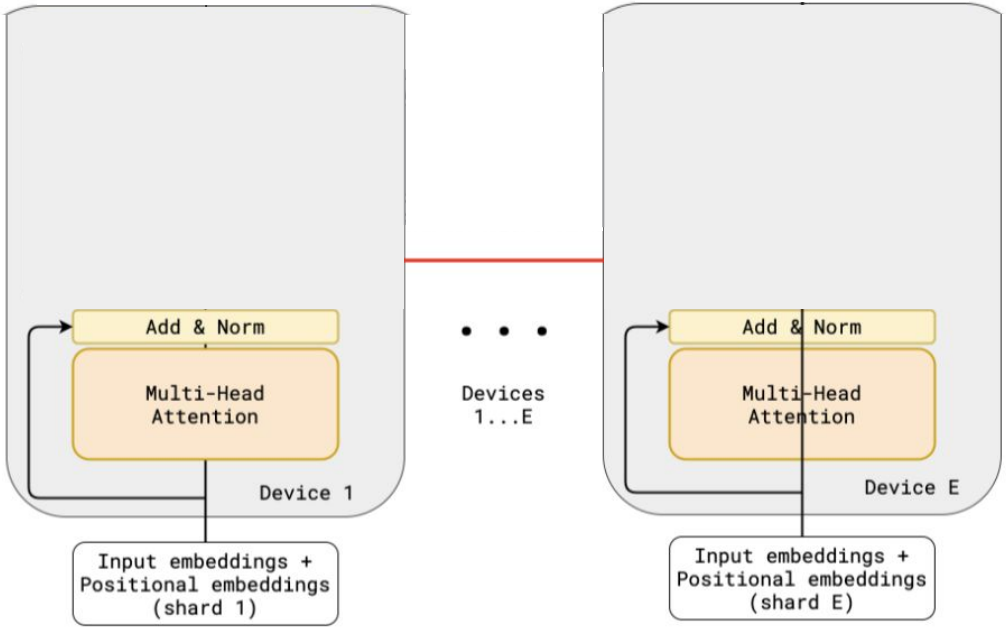


Figure adapted from GShard: Scaling Giant Models with Conditional Computation and Automatic Sharding, Lepikhin et al., 2020.

Model parallelism

Idea. Split the model computations across several devices

- Tensor Parallelism **(TP)**
- Pipeline Parallelism **(PP)**
- Context Parallelism **(CP)**
- **E**xpert **P**arallelism **(EP)**

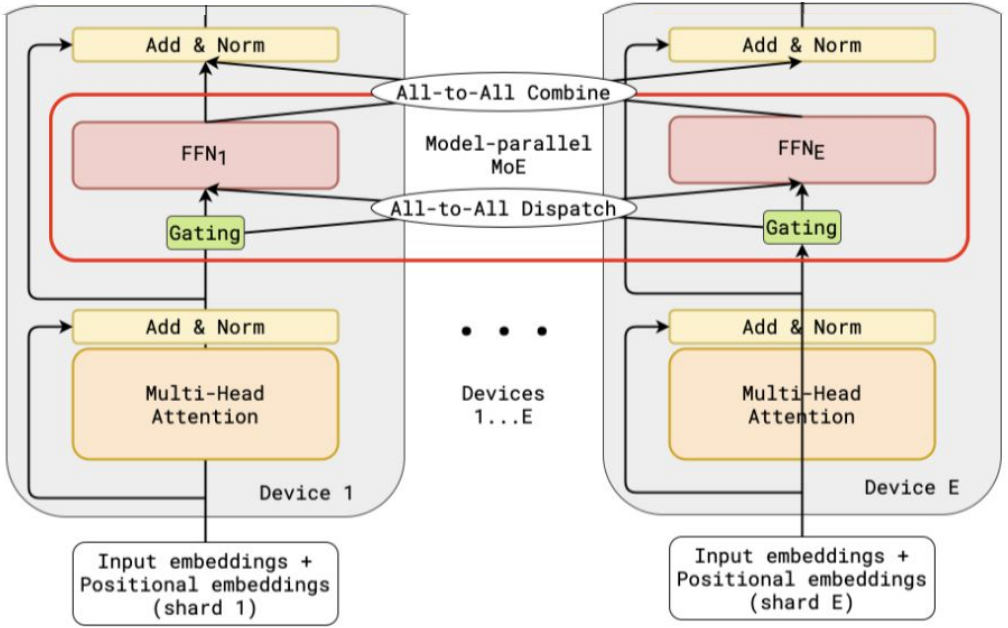


Figure adapted from GShard: Scaling Giant Models with Conditional Computation and Automatic Sharding, Lepikhin et al., 2020.

Model parallelism

Idea. Split the model computations across several devices

- Tensor Parallelism **(TP)**
- Pipeline Parallelism **(PP)**
- Context Parallelism **(CP)**
- **E**xpert **P**arallelism **(EP)**

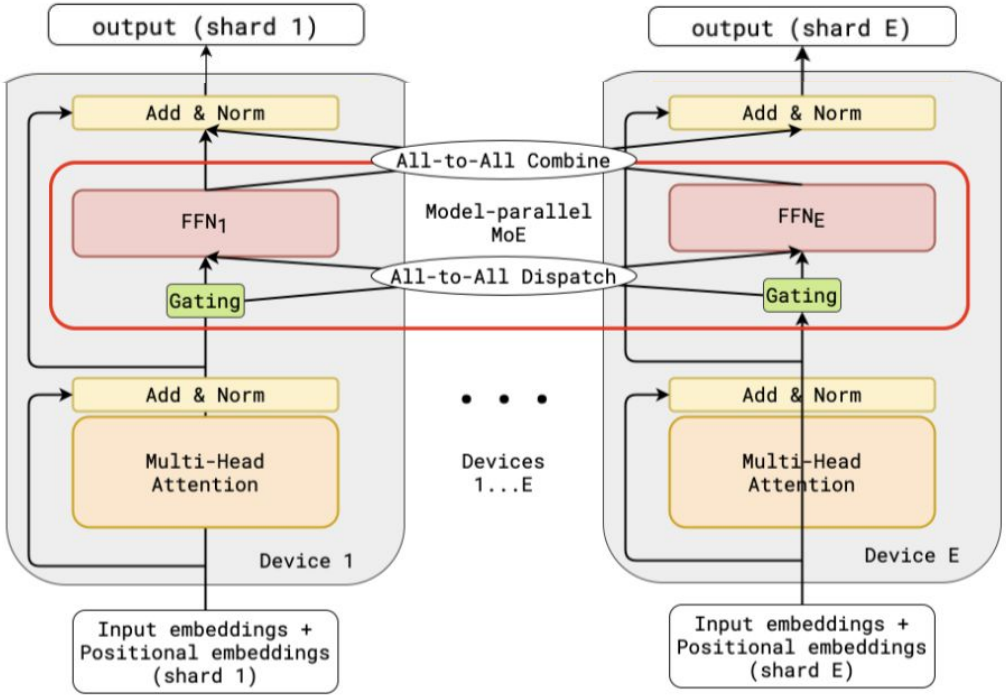


Figure adapted from GShard: Scaling Giant Models with Conditional Computation and Automatic Sharding, Lepikhin et al., 2020.

Discussion

What prevents us from always parallelizing as much as possible?

- Low utilization
- GPUs are expensive
- Communication costs

Summary for parallelism techniques



How do we make it **fit**?

↳ **Distribute** parameters, activations, gradients, optimizer states **across GPUs**

Summary for parallelism techniques

How do we make it **fit**?

- **Distribute** parameters, activations, gradients, optimizer states **across GPUs**



How do we make it **fast**?

- Process different batches in parallel (**data parallelism**)
- Parallelize model computations (**model parallelism**)



Transformers & Large Language Models

Motivation

Flash attention

Distributed training

Inference optimizations

Discussion

Intuition


10 tokens
Generate a story for my teddy bear to fall asleep
10,000 tokens
Chapter One: The Teddy Bear Who Wasn't Quite Sleepy
A long, gentle bedtime story from the secret world of teddy bears.
In a world made of soft things, [...]
1
10,000 tokens
Examine the teddy bear described below. Consider its plush fur, rounded ears, embroidered smile, squishy paws, pastel bow, and comforting shape. Imagine a small bear nestled in a fluffy blanket beside a warm pillow, ready for [...]
10 tokens
This teddy bear is soft, sweet, and cuddly
2

Intuition

10 tokens

```
Generate a story for my
teddy bear to fall asleep
```

```
Chapter One: The Teddy Bear
Who Wasn't Quite Sleepy
```

10,000 tokens

```
of teddy bears.

In a world made of soft
things, [...]
```

1

```
Examine the teddy bear
described below. Consider
its plush fur, rounded
ears, embroidered smile,
squishy paws, pastel bow,
```

```
beside a warm pillow, ready
for [...]
```

10 tokens

```
This teddy bear is soft,
sweet, and cuddly
```

2

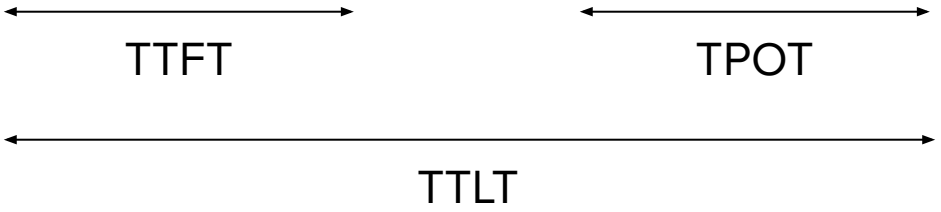
Is (1) or (2) faster?

Overview

Formula.

fast slow

$$T_{\text{request}} = T_{\text{queue}} + T_{\text{prefill}} + (O-1)T_{\text{per output token}}$$



Overview

fast slow

Formula. $T_{\text{request}} = T_{\text{queue}} + T_{\text{prefill}} + (O-1)T_{\text{per output token}}$

TTFT

TPOT

TTLT

Common metrics.

- TTFT = Time To First Token
- TPOT = Time Per Output Token
- TTLT = Time To Last Token

core metrics

Typical contrast in pricing

OpenAI

Model	Input	Output	Prompt caching: Cached input	Prompt caching: 30m write	Prompt caching: 12h write
gpt-6-astra	$10.00	$50.00	$1.00	$12.50	$25.00
gpt-6.1-sol	$2.00	$10.00	$0.10	$2.50	$5.00
gpt-6.1-terra	$1.00	$5.00	$0.05	$1.25	$2.50
gpt-6-luna	$0.10	$0.50	$0.01	$0.125	$0.25

Google

Gemini 4 Argon

`gemini-4-argon`

Try it in Google AI Studio

Our flagship reasoning model, built for complex software engineering, deep mathematical and scientific problem solving, autonomous multi-step agents, and long-decode generation up to 1M tokens.

★ **Note:** Gemini 4 Argon is not available on the Free tier. Every token the model processes is counted and billed, including thinking tokens and internal tool system instructions and function declarations when using grounding.

Standard | Batch | Flex | Priority | Contributor

	Free Tier	Paid Tier, per 1M tokens in USD
Input price	Not available	$2.00 through November 30, 2026. $4.00 starting December 1, 2026.
Output price (including thinking tokens)	Not available	$10.00 through November 30, 2026. $20.00 starting December 1, 2026.
Context caching price	Not available	$0.10 through November 30, 2026. $0.20 starting December 1, 2026. Implicit caching only (explicit caching is not supported).

Anthropic

Model pricing

The following table shows pricing for all Claude models:

Model	Base tokens: Input	Base tokens: Output	Prompt caching: 5m writes	Prompt caching: 1h writes	Prompt caching: Hits and refreshes
Claude Fable 5.1 For demanding reasoning and long-horizon agentic work	$10 / MTok	$50 / MTok	$12.50 / MTok	$20 / MTok	$0.25 / MTok
Claude Opus 5.5 For long-running agentic coding and knowledge work	$4 / MTok	$20 / MTok	$5 / MTok	$8 / MTok	$0.20 / MTok
Claude Sonnet 5.5 The best combination of speed and intelligence	$2 / MTok	$10 / MTok	$2.50 / MTok	$4 / MTok	$0.10 / MTok
Claude Haiku 5.5 For high-volume, latency-sensitive tasks such as classification, extraction, and routing	$0.10 / MTok for prompts up to 100,000 tokens	$0.50 / MTok	$0.125 / MTok	$0.20 / MTok	$0.01 / MTok
	$0.50 / MTok for prompts over 100,000 tokens	$2.50 / MTok	$0.625 / MTok	$1 / MTok	$0.05 / MTok

Screenshots taken from the API pricing pages of OpenAI, Anthropic and Google on October 26, 2026.

Typical contrast in pricing

OpenAI

Model	Input	Output	Prompt caching: Cached input	Prompt caching: 30m write	Prompt caching: 12h write
gpt-6-astra	$10.00	$50.00	$1.00	$12.50	$25.00
gpt-6.1-sol	$2.00	$10.00	$0.10	$2.50	$5.00
gpt-6.1-terra	$1.00	$5.00	$0.05	$1.25	$2.50
gpt-6-luna	$0.10	$0.50	$0.01	$0.125	$0.25

Google

Gemini 4 Argon

`gemini-4-argon`

Try it in Google AI Studio

Our flagship reasoning model, built for complex software engineering, deep mathematical and scientific problem solving, autonomous multi-step agents, and long-decode generation up to 1M tokens.

★ **Note:** Gemini 4 Argon is not available on the Free tier. Every token the model processes is counted and billed, including thinking tokens and internal tool system instructions and function declarations when using grounding.

Standard | Batch | Flex | Priority | Contributor

	Free Tier	Paid Tier, per 1M tokens in USD
Input price	Not available	$2.00 through November 30, 2026. $4.00 starting December 1, 2026.
Output price (including thinking tokens)	Not available	$10.00 through November 30, 2026. $20.00 starting December 1, 2026.
Context caching price	Not available	$0.10 through November 30, 2026. $0.20 starting December 1, 2026. Implicit caching only (explicit caching is not supported).

Anthropic

Model pricing

The following table shows pricing for all Claude models:

Model	Base tokens		Prompt caching		
Name	Input	Output	5m writes	1h writes	Hits and refreshes
Claude Fable 5.1 For demanding reasoning and long-horizon agentic work	$10 / MTok	$50 / MTok	$12.50 / MTok	$20 / MTok	$0.25 / MTok
Claude Opus 5.5 For long-running agentic coding and knowledge work	$4 / MTok	$20 / MTok	$5 / MTok	$8 / MTok	$0.20 / MTok
Claude Sonnet 5.5 The best combination of speed and intelligence	$2 / MTok	$10 / MTok	$2.50 / MTok	$4 / MTok	$0.10 / MTok
Claude Haiku 5.5 For high-volume, latency-sensitive tasks such as classification, extraction, and routing	$0.10 / MTok for prompts up to 100,000 tokens	$0.50 / MTok	$0.125 / MTok	$0.20 / MTok	$0.01 / MTok
	$0.50 / MTok for prompts over 100,000 tokens	$2.50 / MTok	$0.625 / MTok	$1 / MTok	$0.05 / MTok

Screenshots taken from the API pricing pages of OpenAI, Anthropic and Google on October 26, 2026.

Optimization objective

Wishes. We want the system to be:

- Fast → $\mathrm{TTFT} < \tau_1$, $\mathrm{TPOT} < \tau_2$
- High quality → $\mathrm{Quality} > Q_{\min}$
- Cheap → $\text{cost per generated token} < C_{\max}$

Definitions

Computational throughput.

How much computation done per second?

Technical Specifications	
	H200 SXM[1]
FP64	34 TFLOPS
FP64 Tensor Core	67 TFLOPS
FP32	67 TFLOPS
TF32 Tensor Core[2]	989 TFLOPS
BFLOAT16 Tensor Core[2]	1,979 TFLOPS
FP16 Tensor Core[2]	1,979 TFLOPS
FP8 Tensor Core[2]	3,958 TFLOPS
INT8 Tensor Core[2]	3,958 TFLOPS
GPU Memory	141GB
GPU Memory Bandwidth	4.8TB/s
Decoders	7 NVDEC 7 JPEG
Confidential Computing	Supported
Max Thermal Design Power (TDP)	Up to 700W (configurable)
Multi-Instance GPUs	Up to 7 MIGs @18GB each
Form Factor	SXM
Interconnect	NVIDIA NVLink: 900GB/s PCIe Gen5: 128GB/s
Server Options	NVIDIA HGX™ H200 partner and NVIDIA-Certified Systems™ with 4 or 8 GPUs
NVIDIA AI Enterprise	Add-on

Screenshot taken from NVIDIA H200 Tensor Core GPU, NVIDIA website.

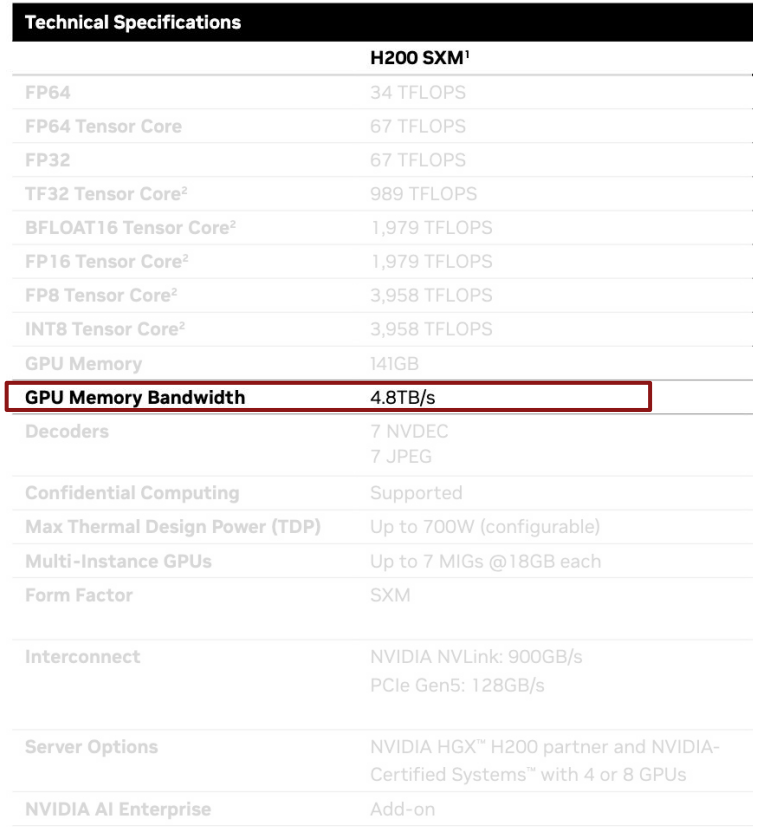
Definitions

Computational throughput.

How much computation done per second?

Memory bandwidth.

How much data transferred per second?



Technical Specifications	
	H200 SXM[1]
FP64	34 TFLOPS
FP64 Tensor Core	67 TFLOPS
FP32	67 TFLOPS
TF32 Tensor Core[2]	989 TFLOPS
BFLOAT16 Tensor Core[2]	1,979 TFLOPS
FP16 Tensor Core[2]	1,979 TFLOPS
FP8 Tensor Core[2]	3,958 TFLOPS
INT8 Tensor Core[2]	3,958 TFLOPS
GPU Memory	141GB
GPU Memory Bandwidth	4.8TB/s
Decoders	7 NVDEC 7 JPEG
Confidential Computing	Supported
Max Thermal Design Power (TDP)	Up to 700W (configurable)
Multi-Instance GPUs	Up to 7 MIGs @18GB each
Form Factor	SXM
Interconnect	NVIDIA NVLink: 900GB/s PCIe Gen5: 128GB/s
Server Options	NVIDIA HGX™ H200 partner and NVIDIA-Certified Systems™ with 4 or 8 GPUs
NVIDIA AI Enterprise	Add-on

Screenshot taken from NVIDIA H200 Tensor Core GPU, NVIDIA website.

Definitions

Computational throughput.

How much computation done per second?

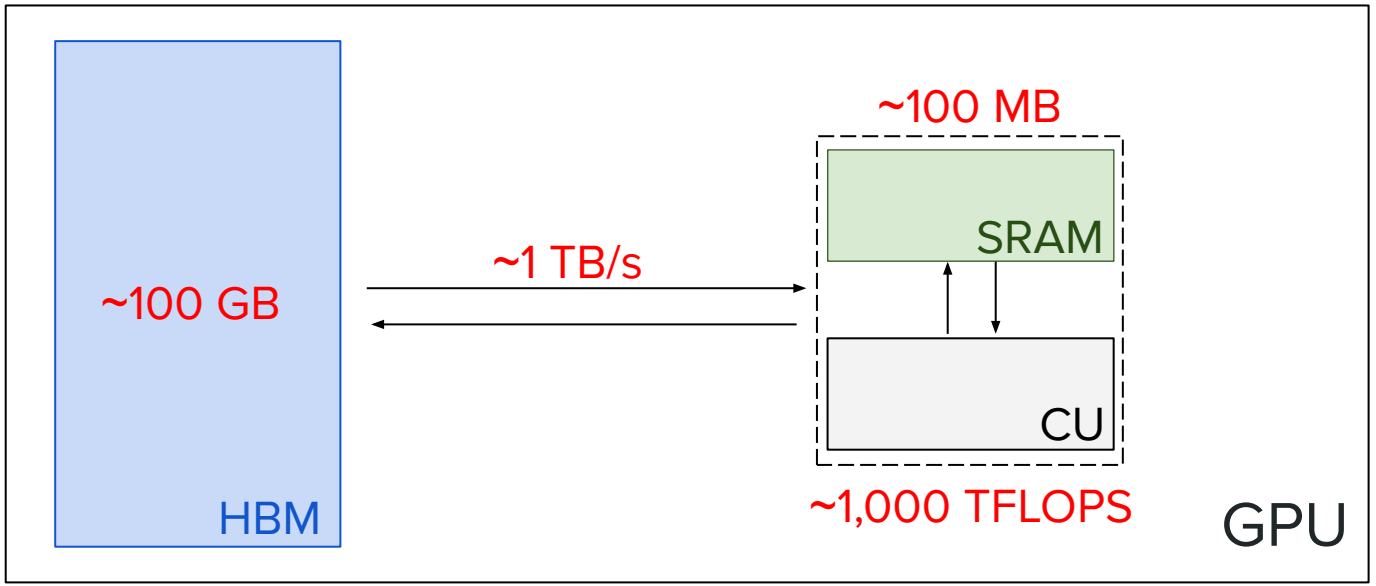
Memory bandwidth.

How much data transferred per second?

Arithmetic intensity.

How much computation done per byte? $\longrightarrow \frac{\text{computations done}}{\text{memory moved}}$

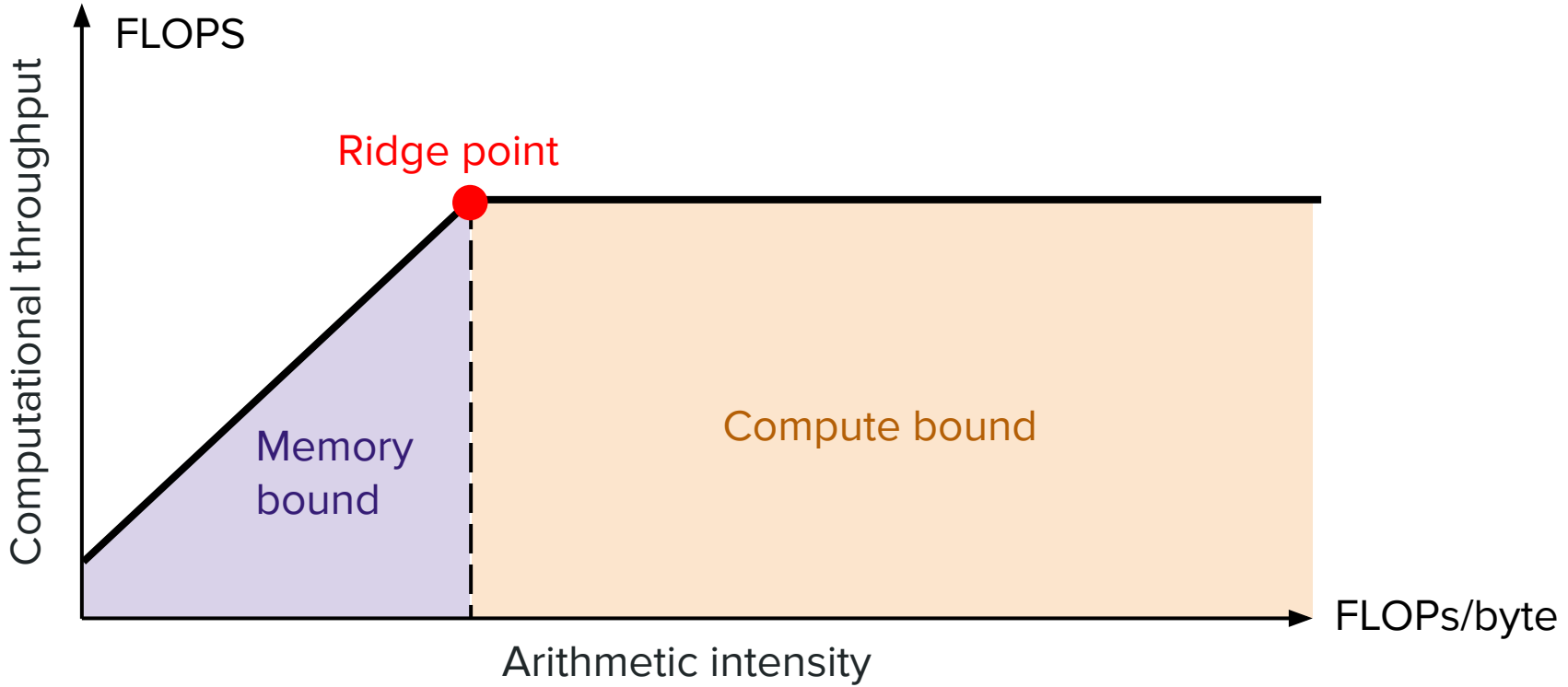
GPU orders of magnitude



Orders of magnitude derived from NVIDIA H200 Tensor Core GPU, NVIDIA website.

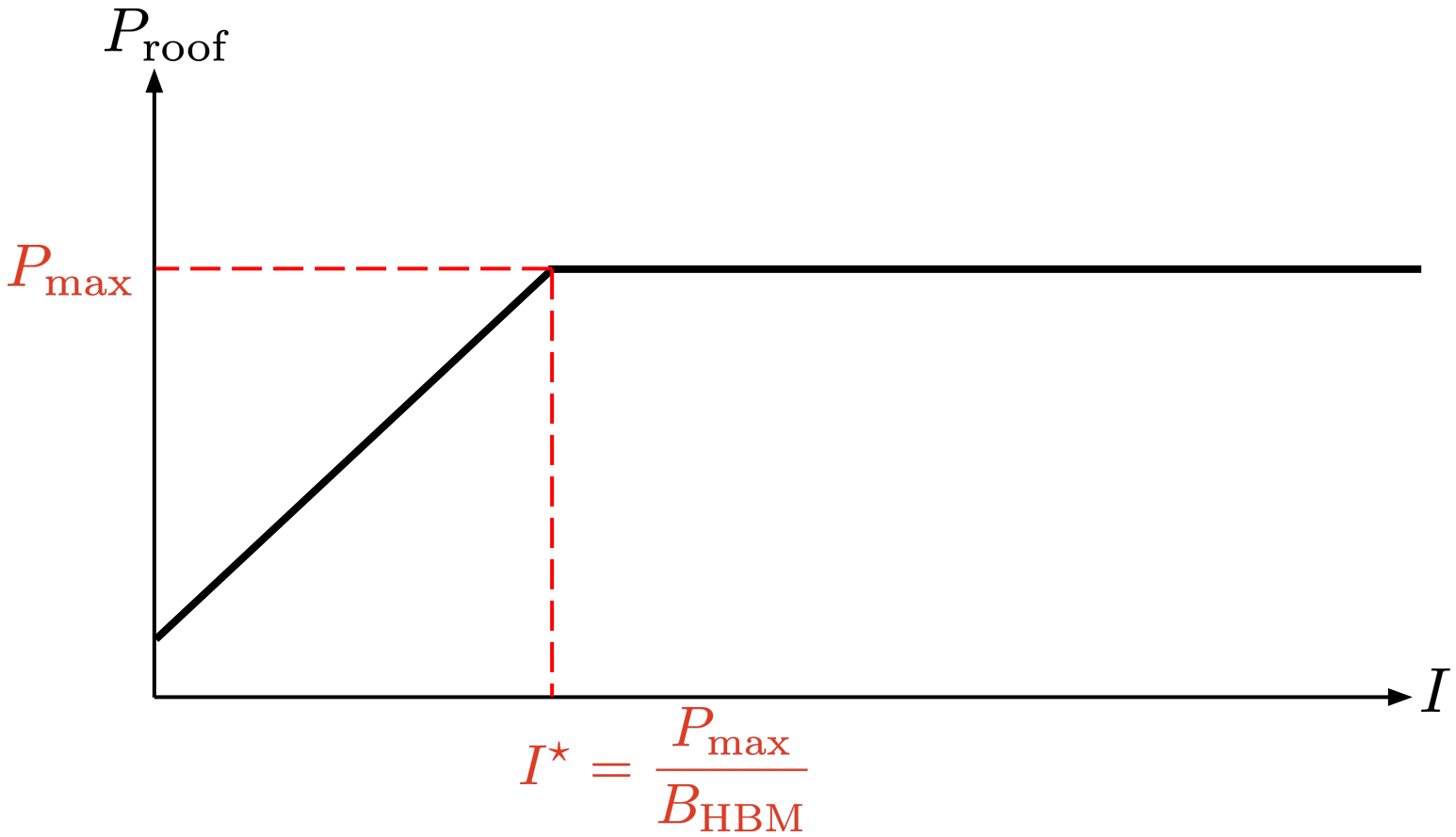
Compute vs. memory

"work done per time"



"work done per data transferred"

Compute vs. memory



Attention-based tricks

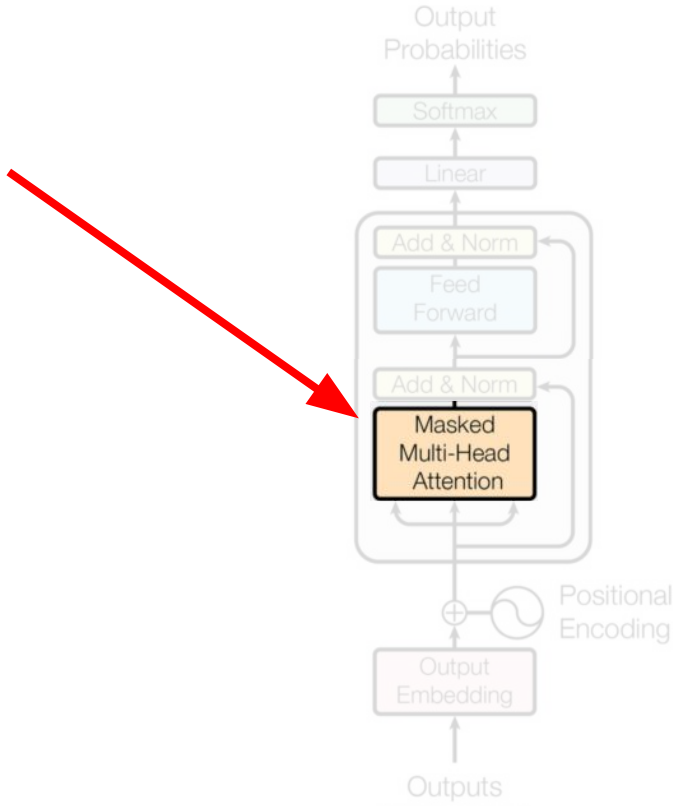


Figure adapted from Attention Is All You Need, Vaswani et al., 2017.

Reusing computations with KV caching

Motivation. New token needs to interact with all previous tokens.

$$\text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Reusing computations with KV caching

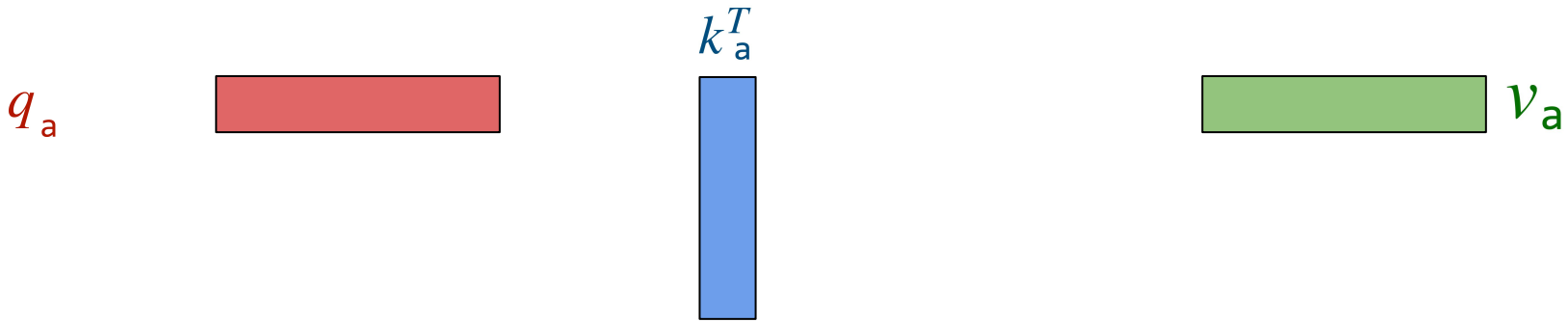
Motivation. New token needs to interact with all previous tokens.

Idea. Keep keys and values in a cache.

Reusing computations with KV caching

Motivation. New token needs to interact with all previous tokens.

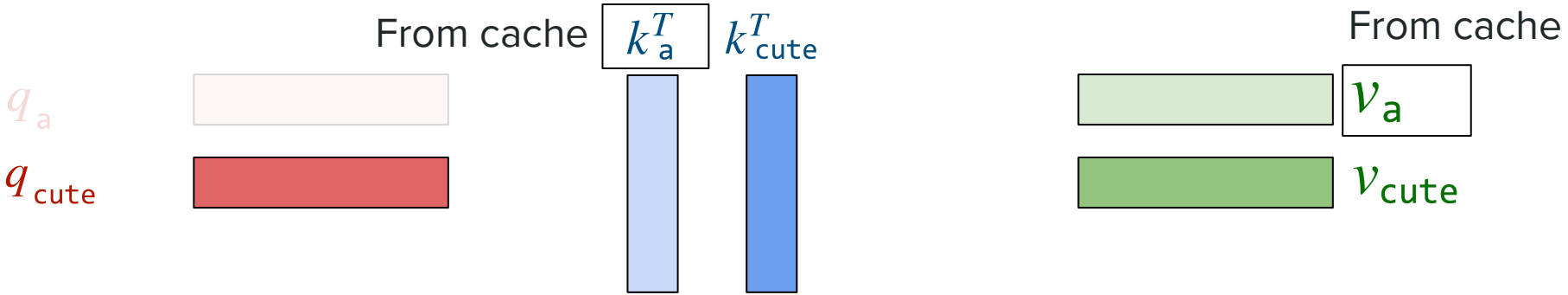
Idea. Keep keys and values in a cache.



Reusing computations with KV caching

Motivation. New token needs to interact with all previous tokens.

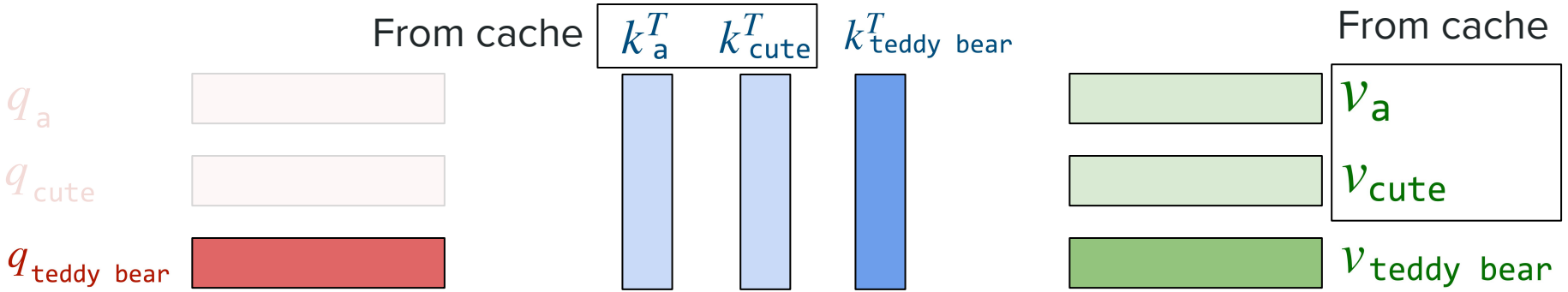
Idea. Keep keys and values in a cache.



Reusing computations with KV caching

Motivation. New token needs to interact with all previous tokens.

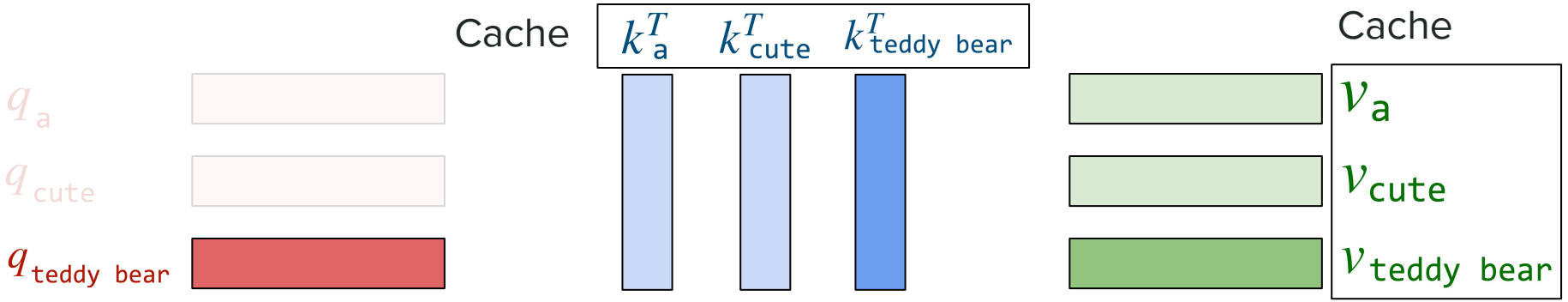
Idea. Keep keys and values in a cache.



Reusing computations with KV caching

Motivation. New token needs to interact with all previous tokens.

Idea. Keep keys and values in a cache.



KV caching = **K**ey-**V**alue **Caching**

Typical cached pricing

OpenAI

Model	Input	Output	Prompt caching: Cached input	Prompt caching: 30m write	Prompt caching: 12h write
gpt-6-astra	$10.00	$50.00	$1.00	$12.50	$25.00
gpt-6.1-sol	$2.00	$10.00	$0.10	$2.50	$5.00
gpt-6.1-terra	$1.00	$5.00	$0.05	$1.25	$2.50
gpt-6-luna	$0.10	$0.50	$0.01	$0.125	$0.25

Google

Gemini 4 Argon

`gemini-4-argon`

Try it in Google AI Studio

Our flagship reasoning model, built for complex software engineering, deep mathematical and scientific problem solving, autonomous multi-step agents, and long-decode generation up to 1M tokens.

★ **Note:** Gemini 4 Argon is not available on the Free tier. Every token the model processes is counted and billed, including thinking tokens and internal tool system instructions and function declarations when using grounding.

Standard | Batch | Flex | Priority | Contributor

	Free Tier	Paid Tier, per 1M tokens in USD
Input price	Not available	$2.00 through November 30, 2026. $4.00 starting December 1, 2026.
Output price (including thinking tokens)	Not available	$10.00 through November 30, 2026. $20.00 starting December 1, 2026.
Context caching price	Not available	$0.10 through November 30, 2026. $0.20 starting December 1, 2026. Implicit caching only (explicit caching is not supported).

Anthropic

Model pricing

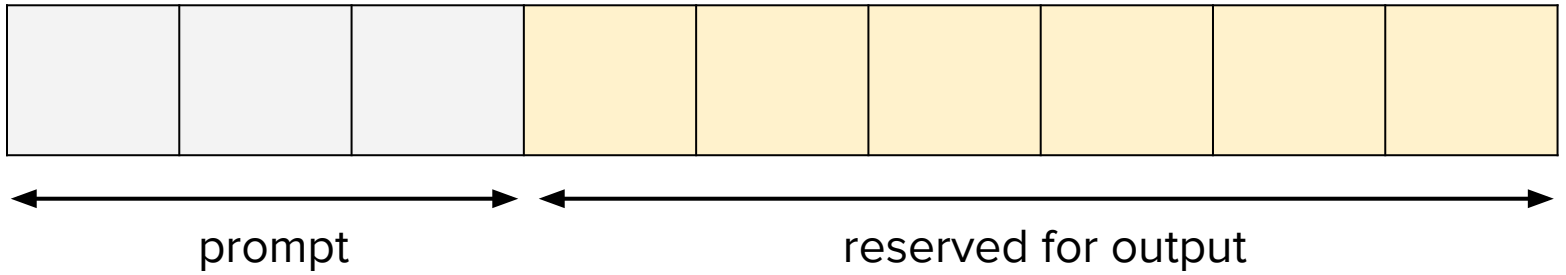
The following table shows pricing for all Claude models:

Model	Base tokens		Prompt caching		
Name	Input	Output	5m writes	1h writes	Hits and refreshes
Claude Fable 5.1 For demanding reasoning and long-horizon agentic work	$10 / MTok	$50 / MTok	$12.50 / MTok	$20 / MTok	$0.25 / MTok
Claude Opus 5.5 For long-running agentic coding and knowledge work	$4 / MTok	$20 / MTok	$5 / MTok	$8 / MTok	$0.20 / MTok
Claude Sonnet 5.5 The best combination of speed and intelligence	$2 / MTok	$10 / MTok	$2.50 / MTok	$4 / MTok	$0.10 / MTok
Claude Haiku 5.5 For high-volume, latency-sensitive tasks such as classification, extraction, and routing	$0.10 / MTok for prompts up to 100,000 tokens	$0.50 / MTok	$0.125 / MTok	$0.20 / MTok	$0.01 / MTok
	$0.50 / MTok for prompts over 100,000 tokens	$2.50 / MTok	$0.625 / MTok	$1 / MTok	$0.05 / MTok

Screenshots taken from the API pricing pages of OpenAI, Anthropic and Google on October 26, 2026.

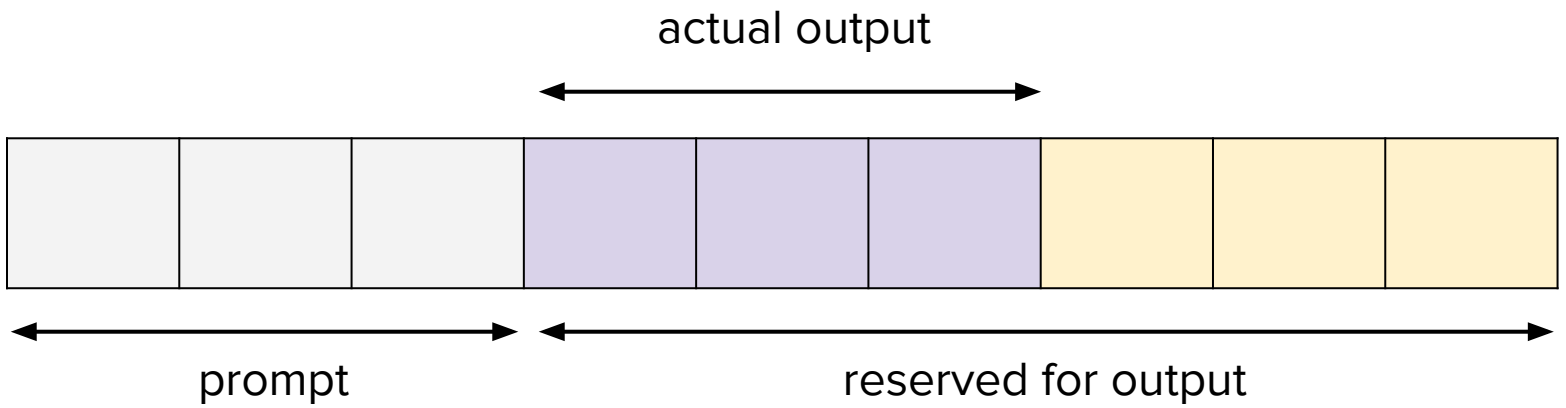
KV cache storage efficiency

Observation. Naive management



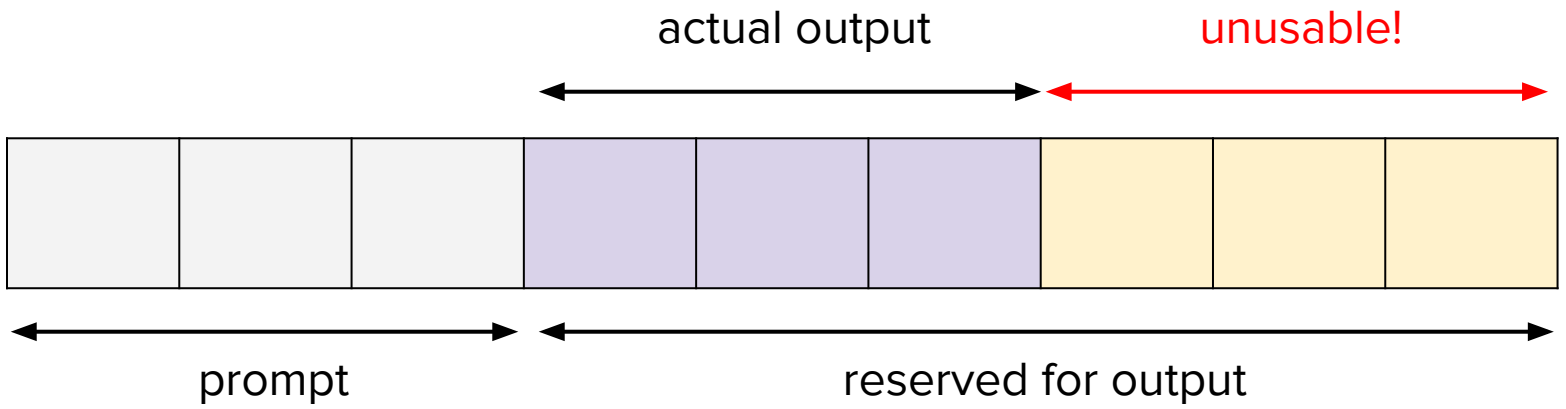
KV cache storage efficiency

Observation. Naive management

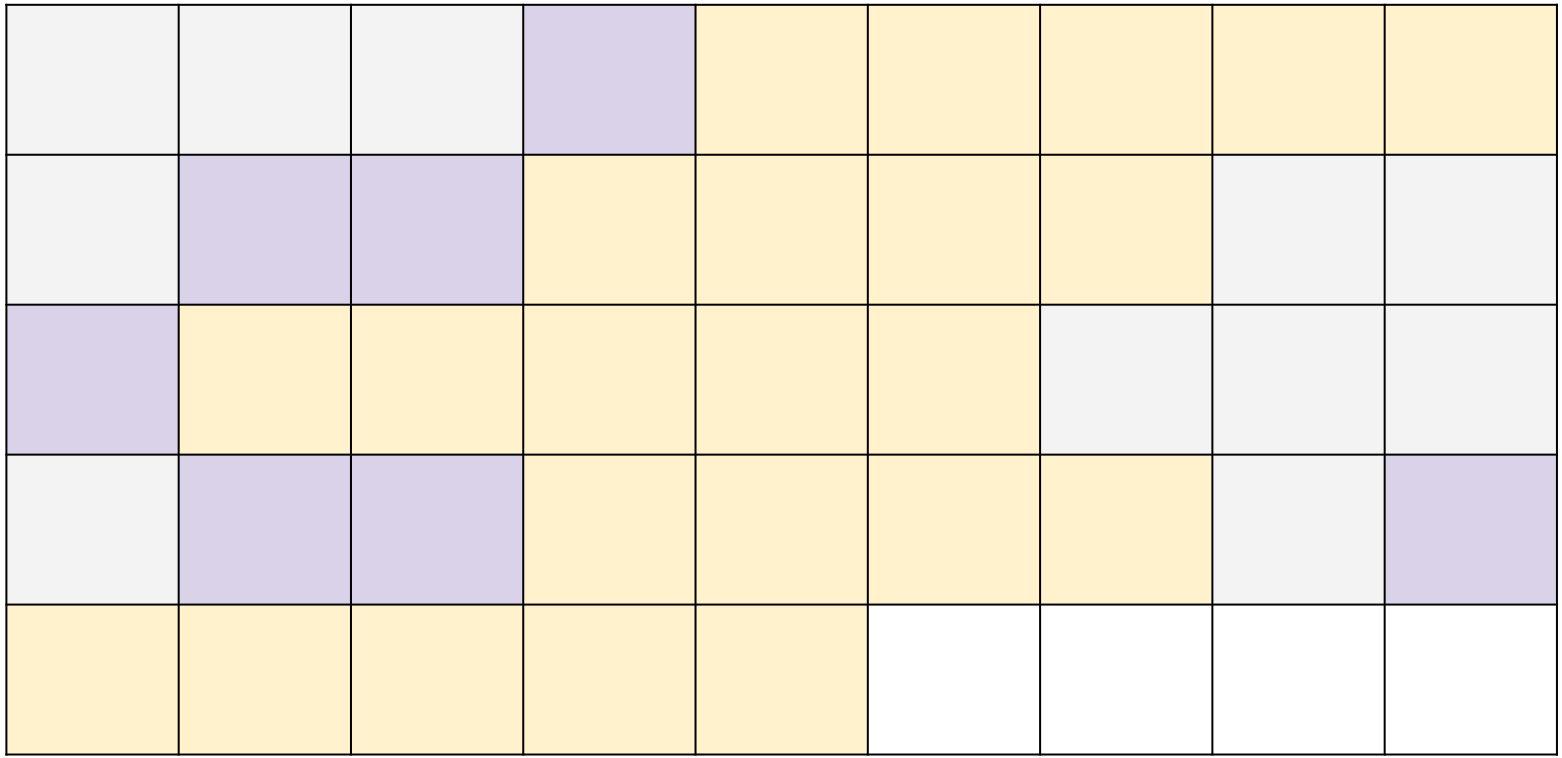


KV cache storage efficiency

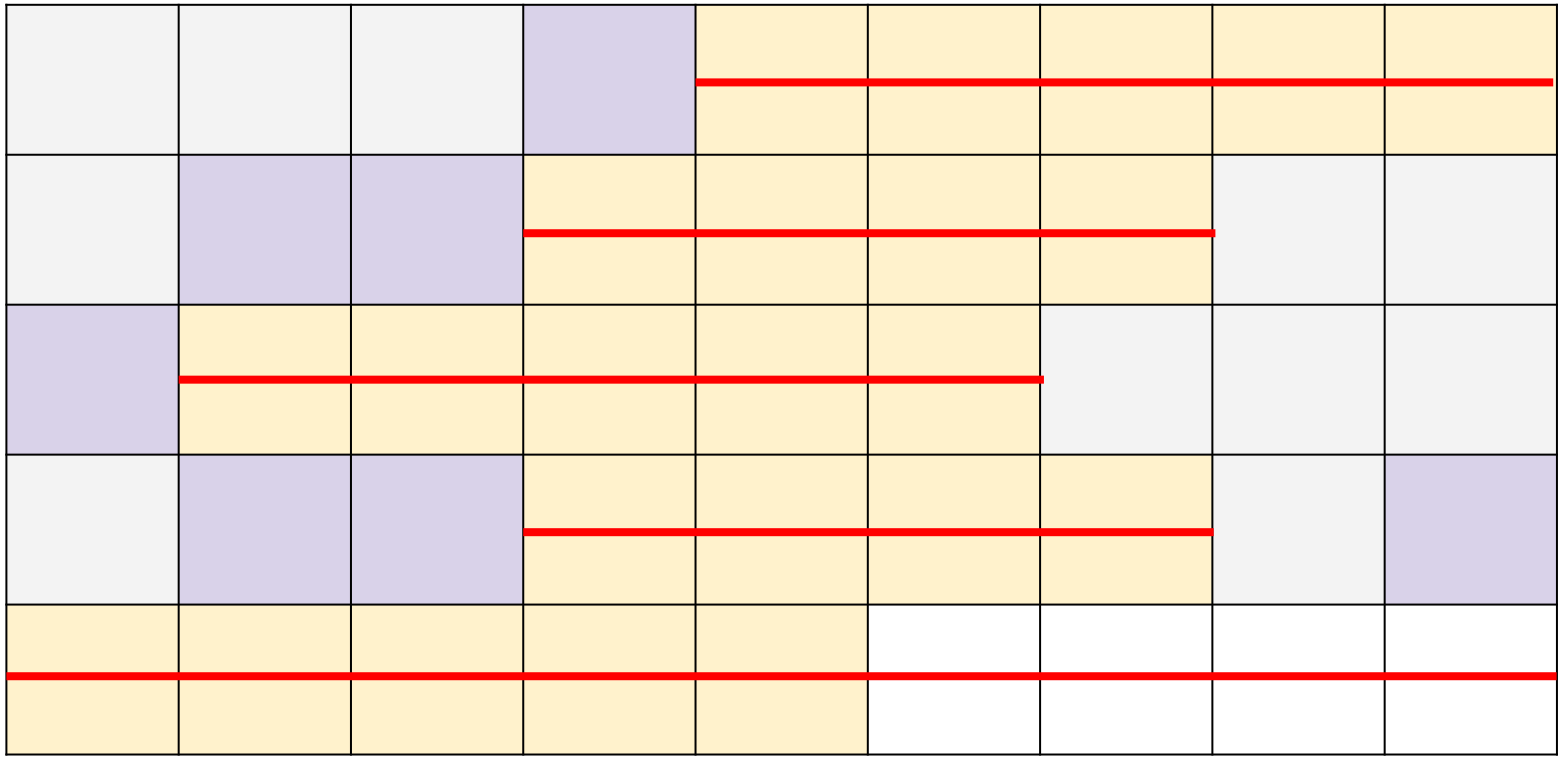
Observation. Naive management = fragmentation everywhere!



KV cache storage efficiency

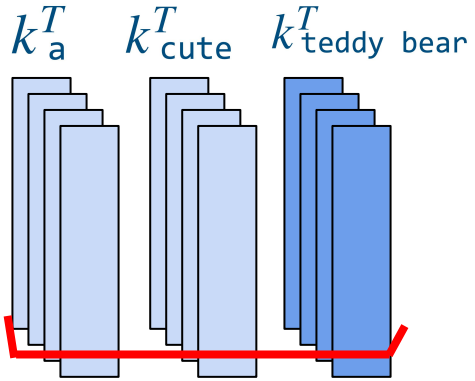


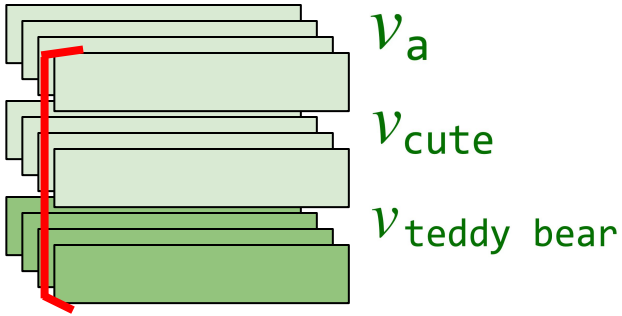
KV cache storage efficiency



unusable!

KV cache storage efficiency

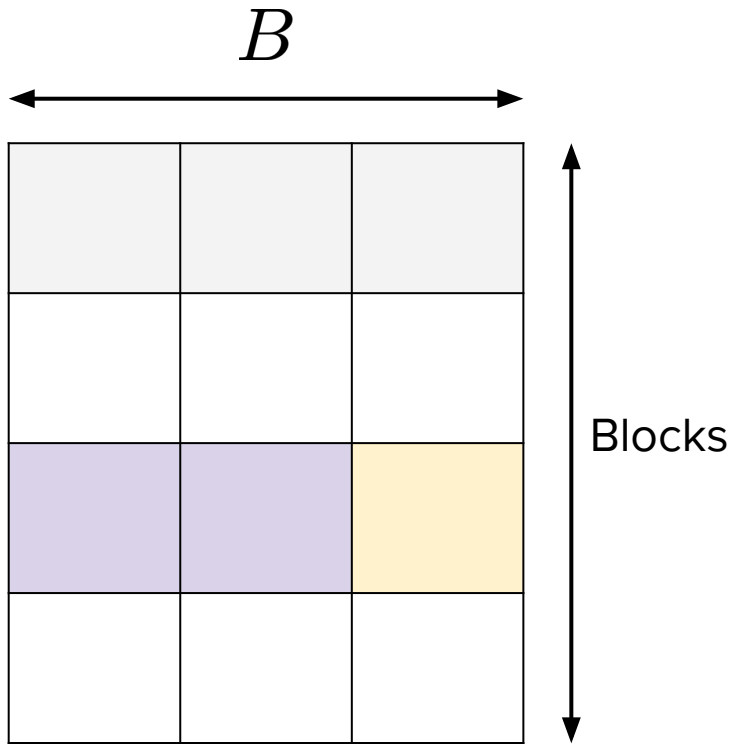




❌ Vectors fragmented

KV cache storage efficiency

Solution. Lookup table!



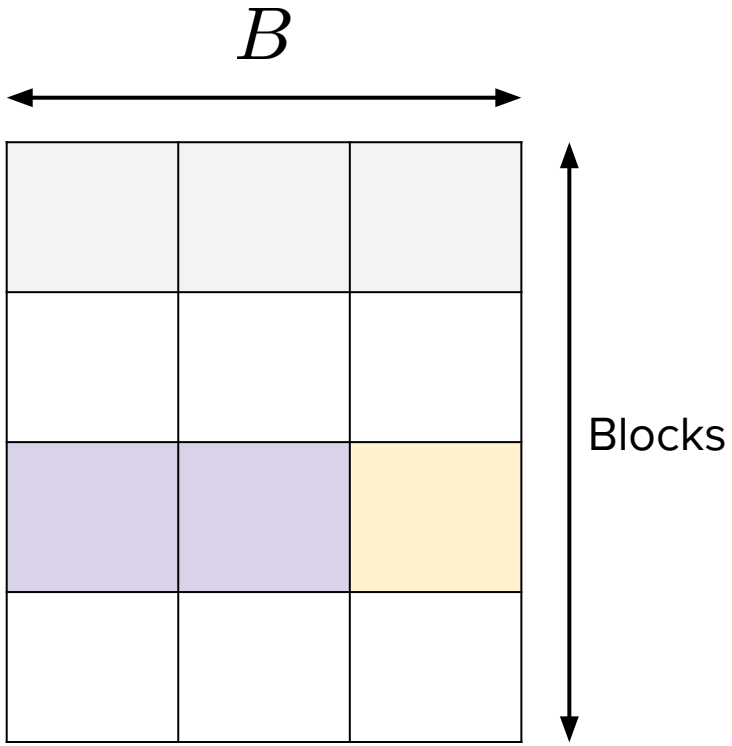
Efficient Memory Management for Large Language Model Serving with PagedAttention, Kwon et al., 2023.

KV cache storage efficiency

Solution. Lookup table!

Trade-offs. Block size:

- Low B: less fragmentation, but more computational overhead
- High B: more waste, less overhead



Efficient Memory Management for Large Language Model Serving with PagedAttention, Kwon et al., 2023.

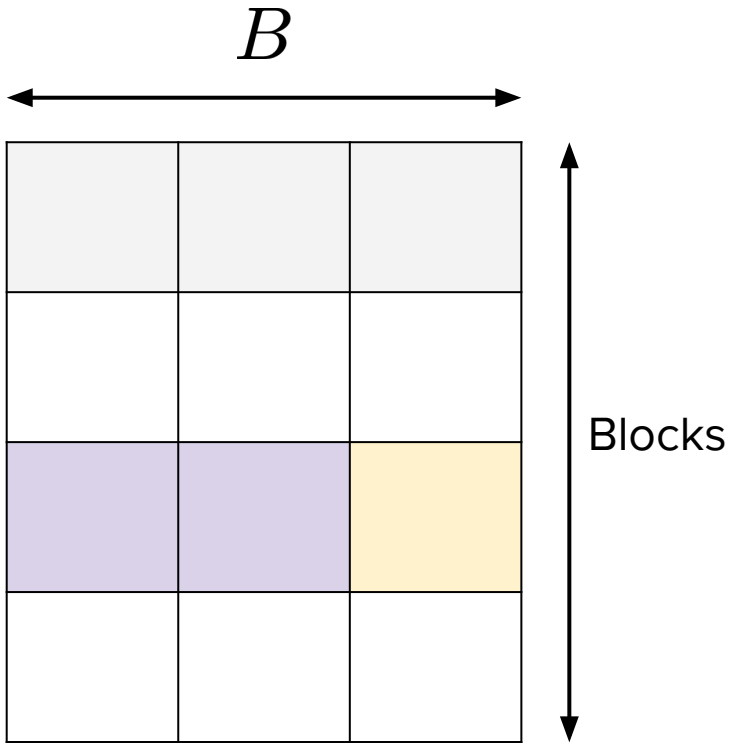
KV cache storage efficiency

Solution. Lookup table!

Trade-offs. Block size:

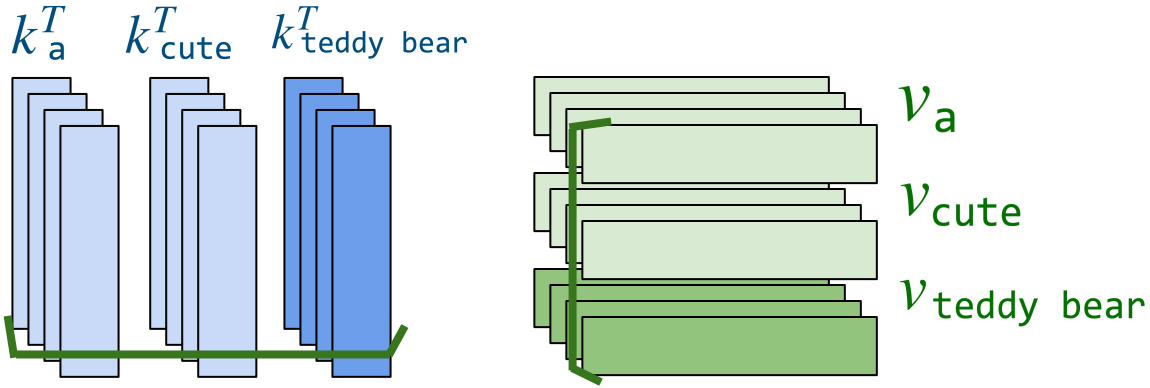
- Low B: less fragmentation, but more computational overhead
- High B: more waste, less overhead

Typical value. $B = 16$



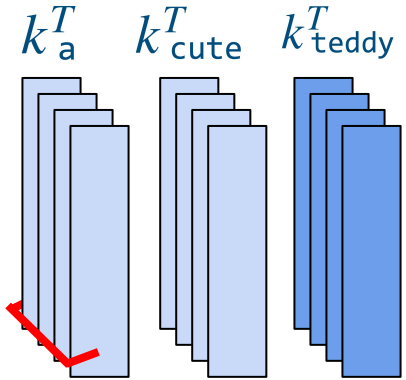
Efficient Memory Management for Large Language Model Serving with PagedAttention, Kwon et al., 2023.

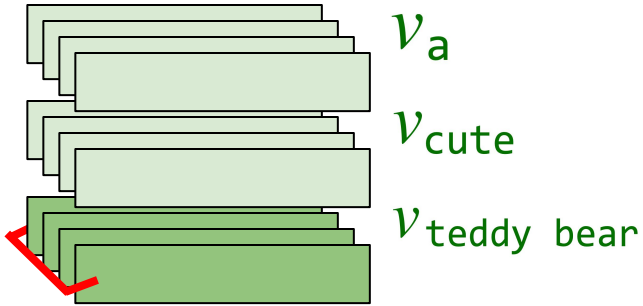
KV cache storage efficiency



✅ Memory management with PagedAttention

Sharing attention heads

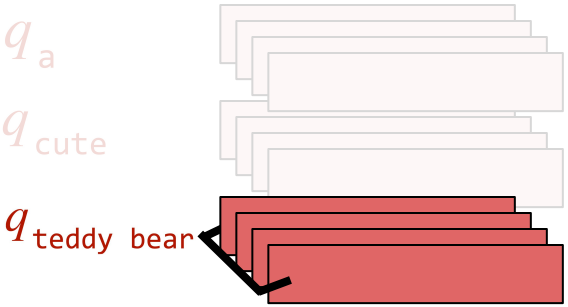




✅ Memory management with PagedAttention

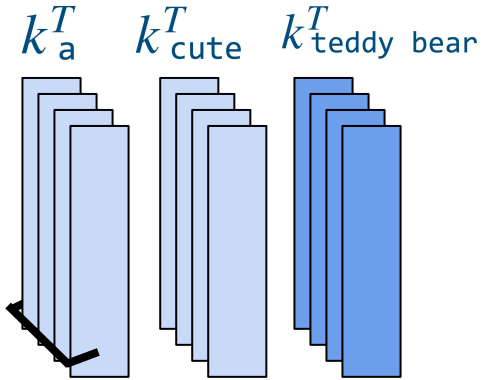
❌ Too many vectors

Sharing attention heads



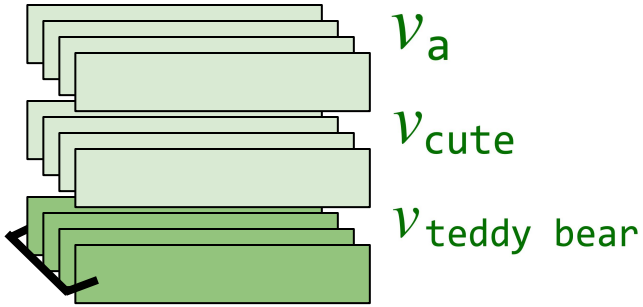
Number of
query heads

h



Number of
key heads

h



Number of
value heads

h

Sharing attention heads

Idea. Share key/value attention heads within groups of queries

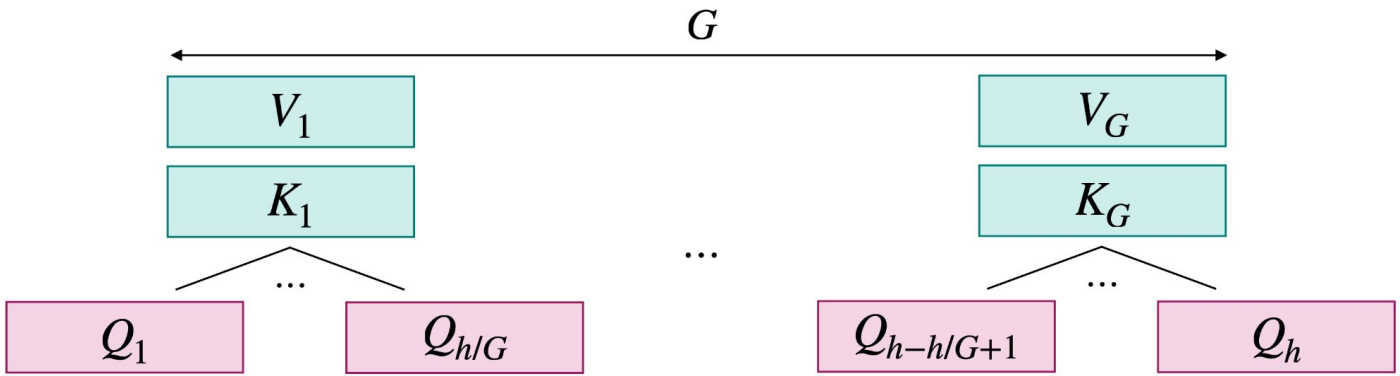


Figure from Super Study Guide: Transformers & Large Language Models, Amidi et al., 2024.

Sharing attention heads

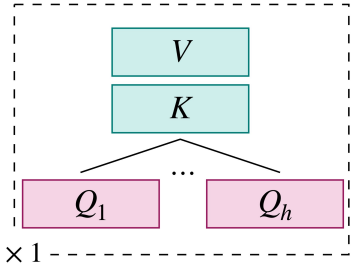
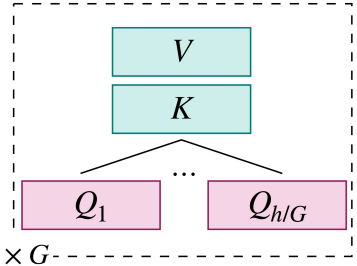
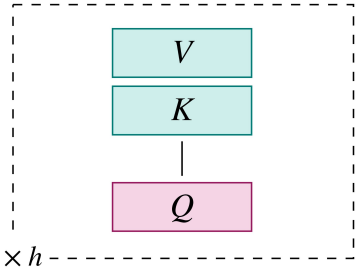
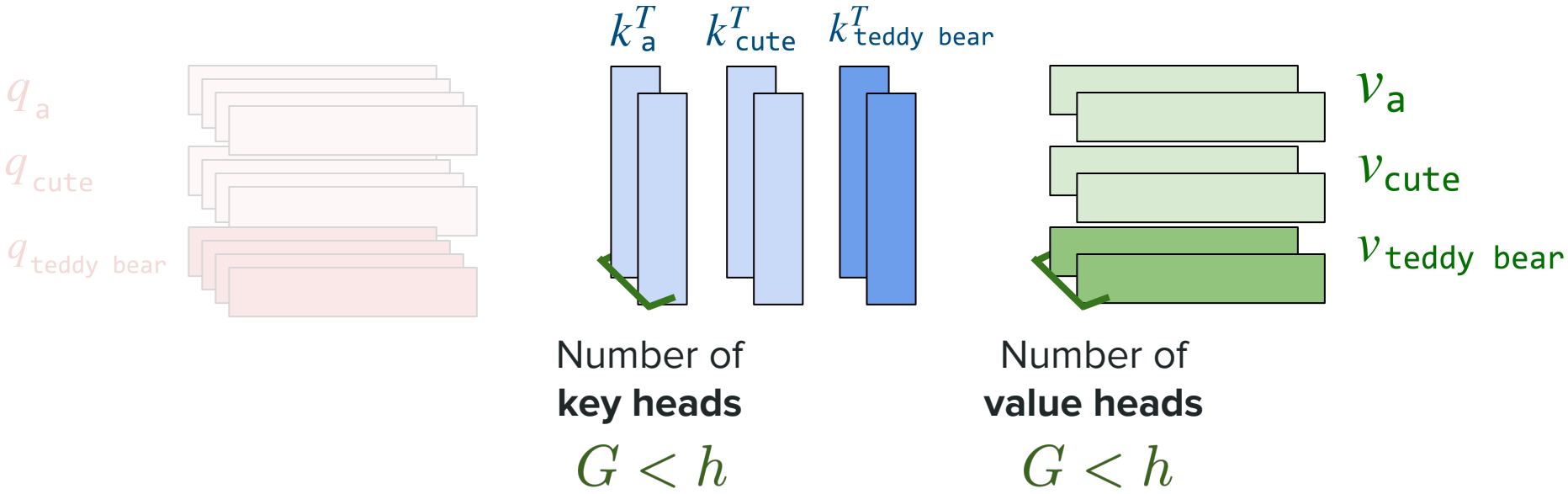
$G = 1$	**M**ulti-**Q**uery **A**ttention (MQA)	
$1 < G < h$	**G**rouped-**Q**uery **A**ttention (GQA)	
$G = h$	**M**ulti-**H**ead **A**ttention (MHA)	

Figure from Super Study Guide: Transformers & Large Language Models, Amidi et al., 2024.

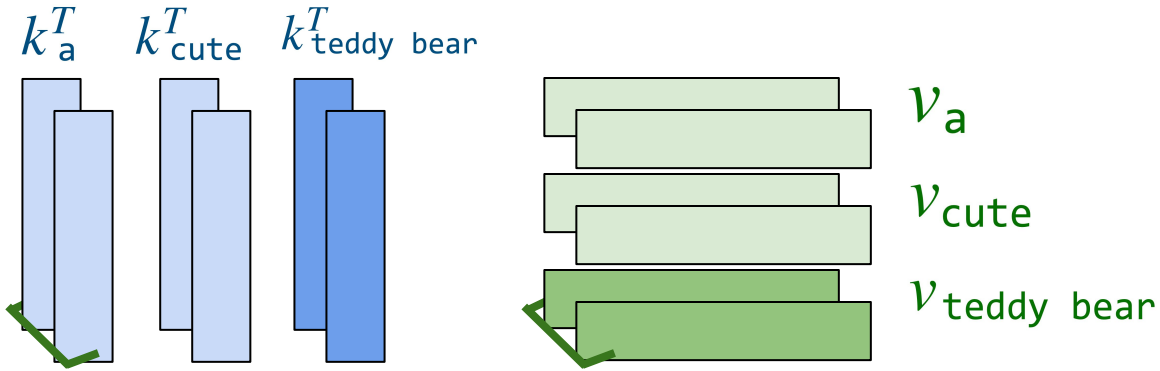
Sharing attention heads



Order of magnitude. h = 24, G = 2 leads to more than 90% savings!

Qwen3.8-Flash-Next model card, Hugging Face, August 2026.

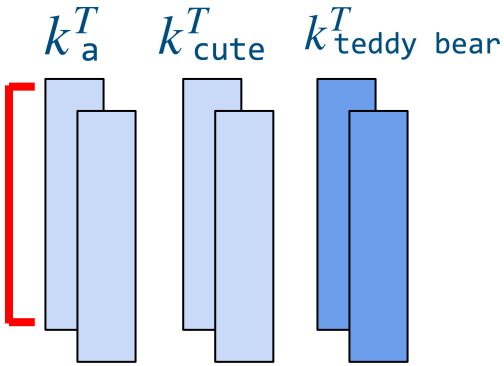
Sharing attention heads

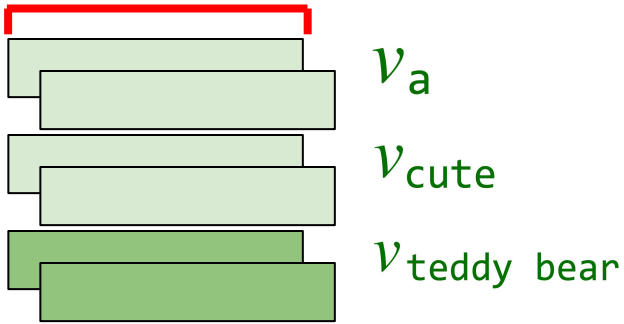


✅ Memory management with PagedAttention

✅ Fewer vectors with GQA

Dimension reduction





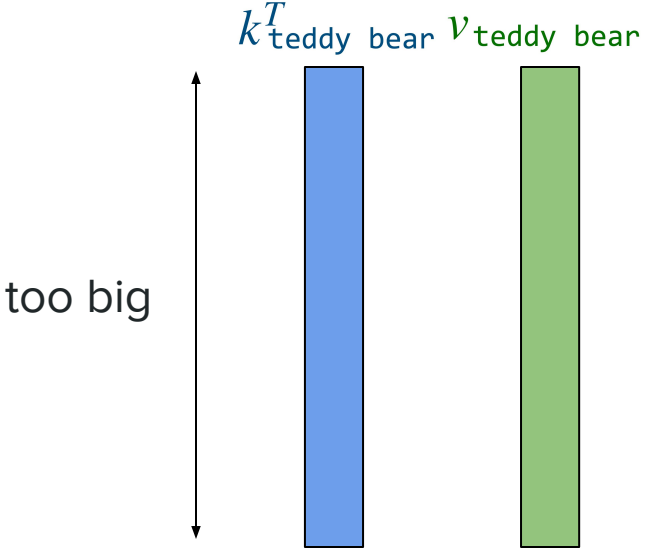
✅ Memory management with PagedAttention

✅ Fewer vectors with GQA

❌ Vectors too long

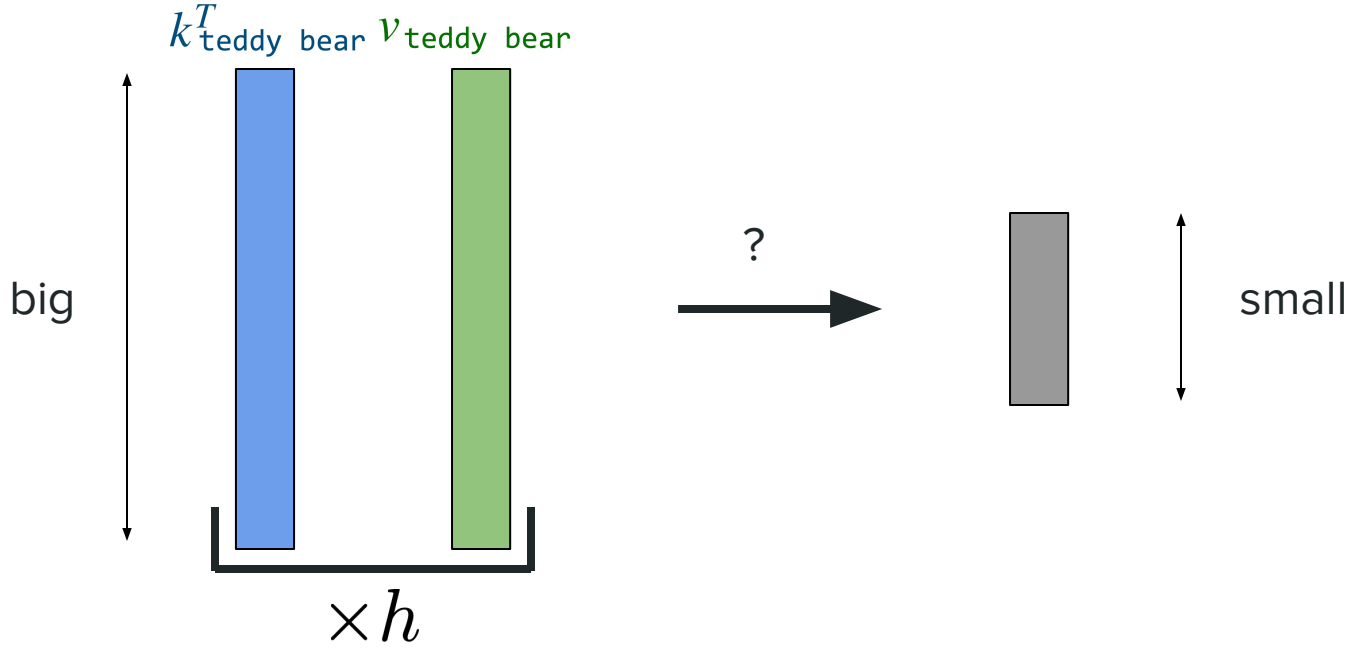
Dimension reduction

Goal. Reduce dimension of K and V stored in memory.



Dimension reduction

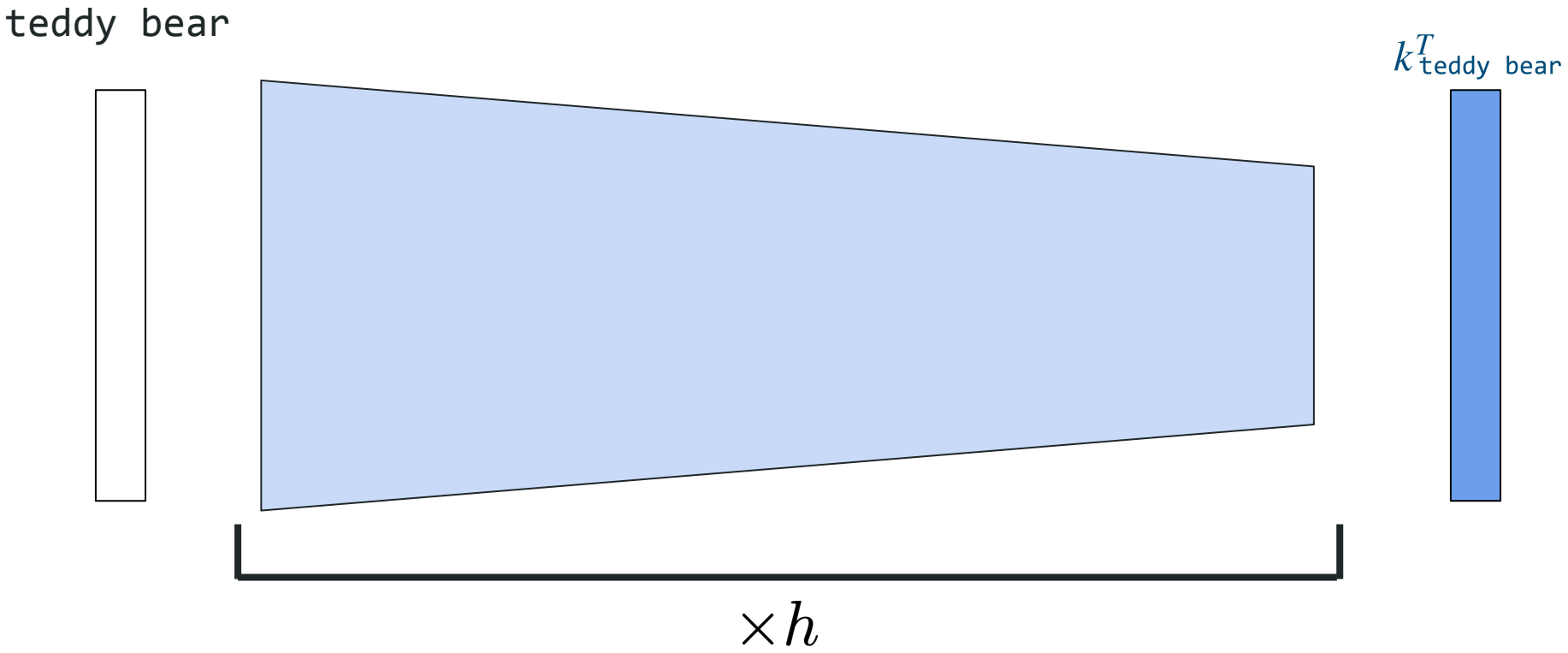
Solution. Store compressed representations instead!



DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model, by DeepSeek, 2024.

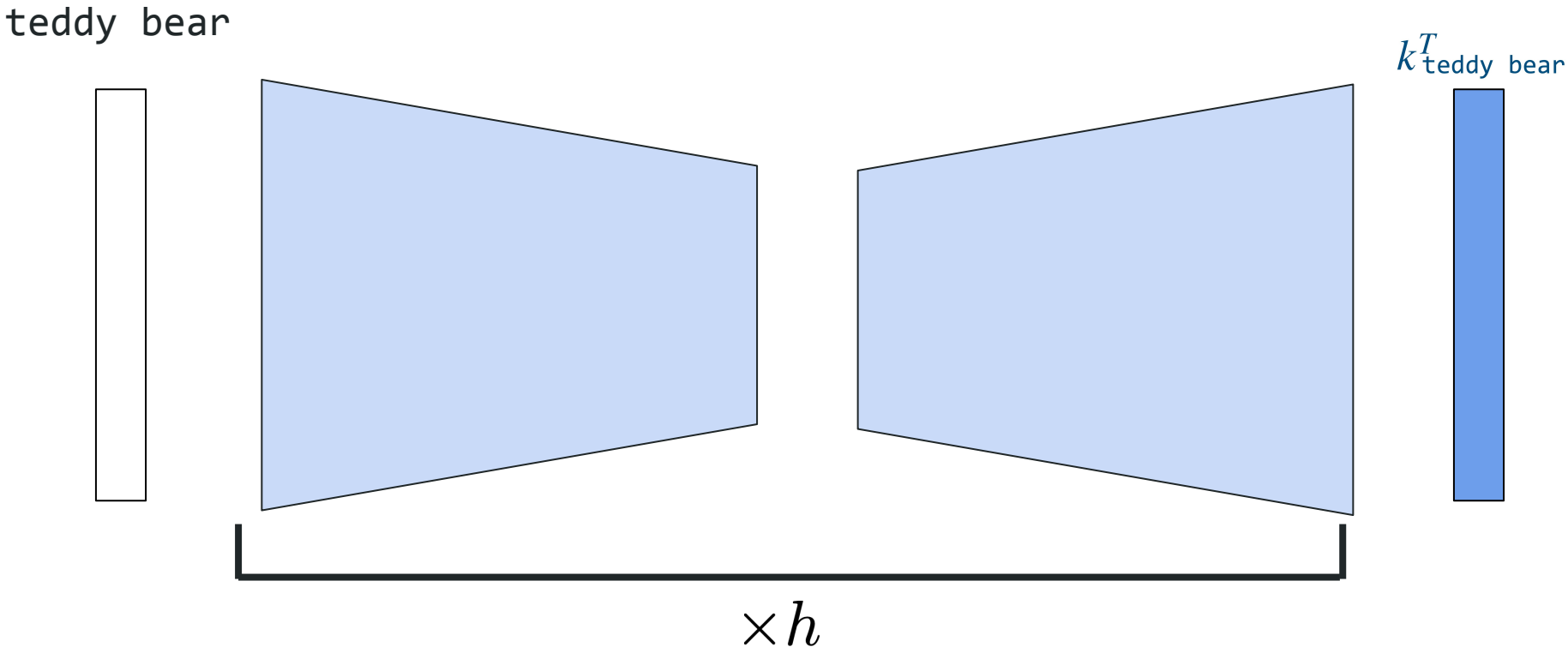
Dimension reduction

BEFORE



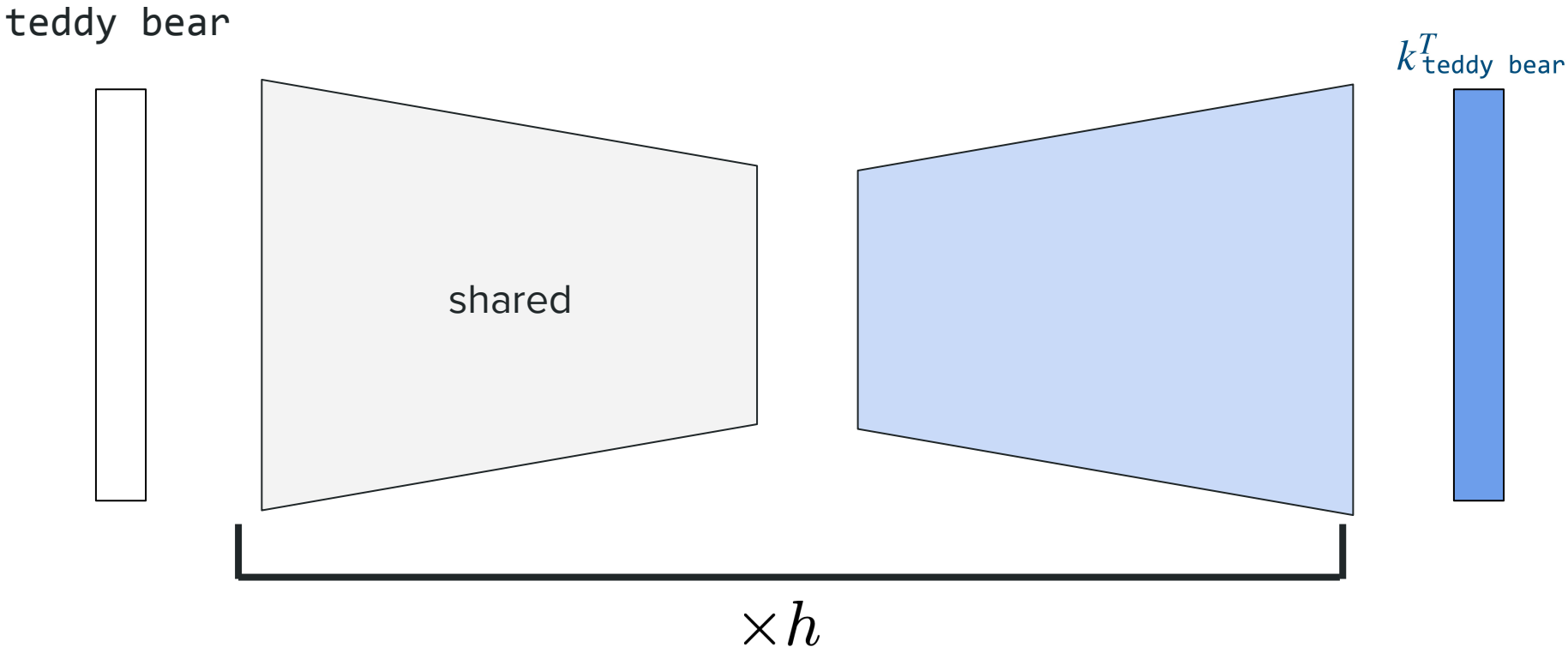
DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model, by DeepSeek, 2024.

Dimension reduction



DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model, by DeepSeek, 2024.

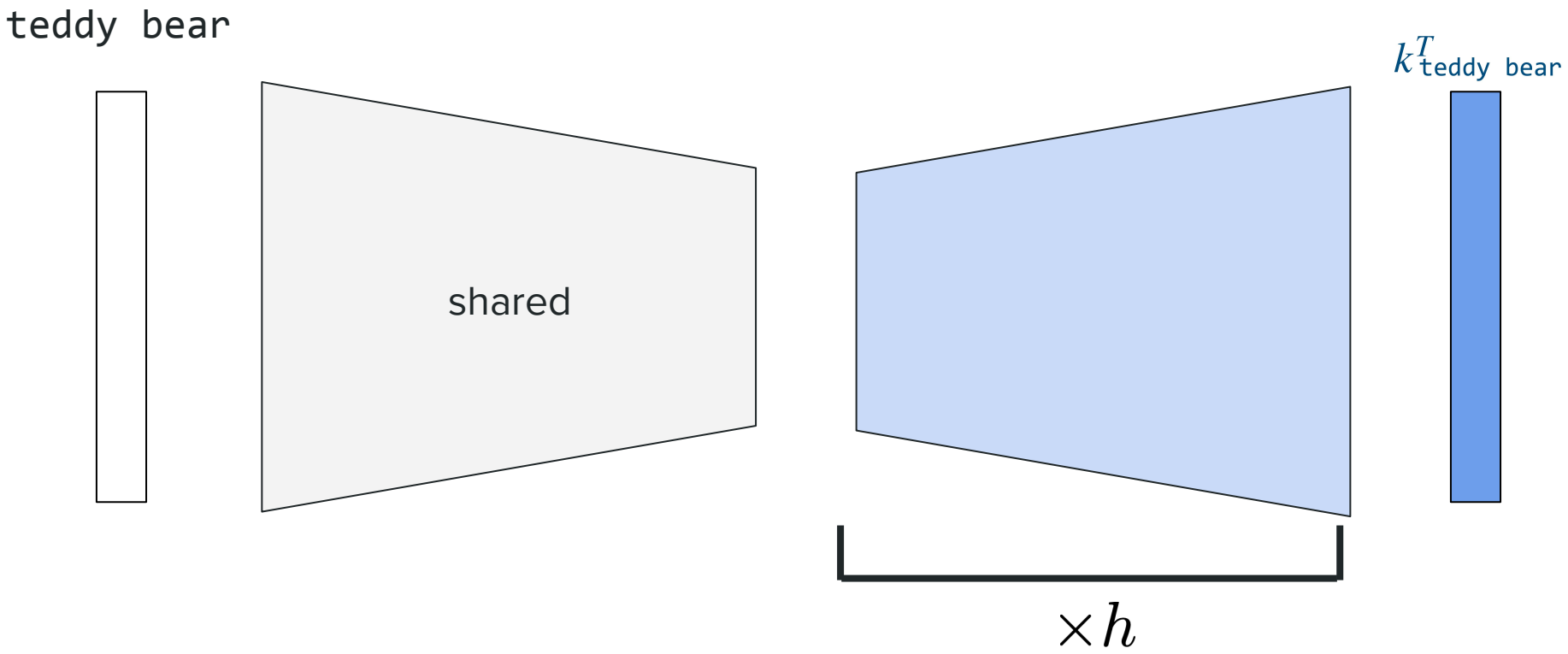
Dimension reduction



DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model, by DeepSeek, 2024.

Dimension reduction

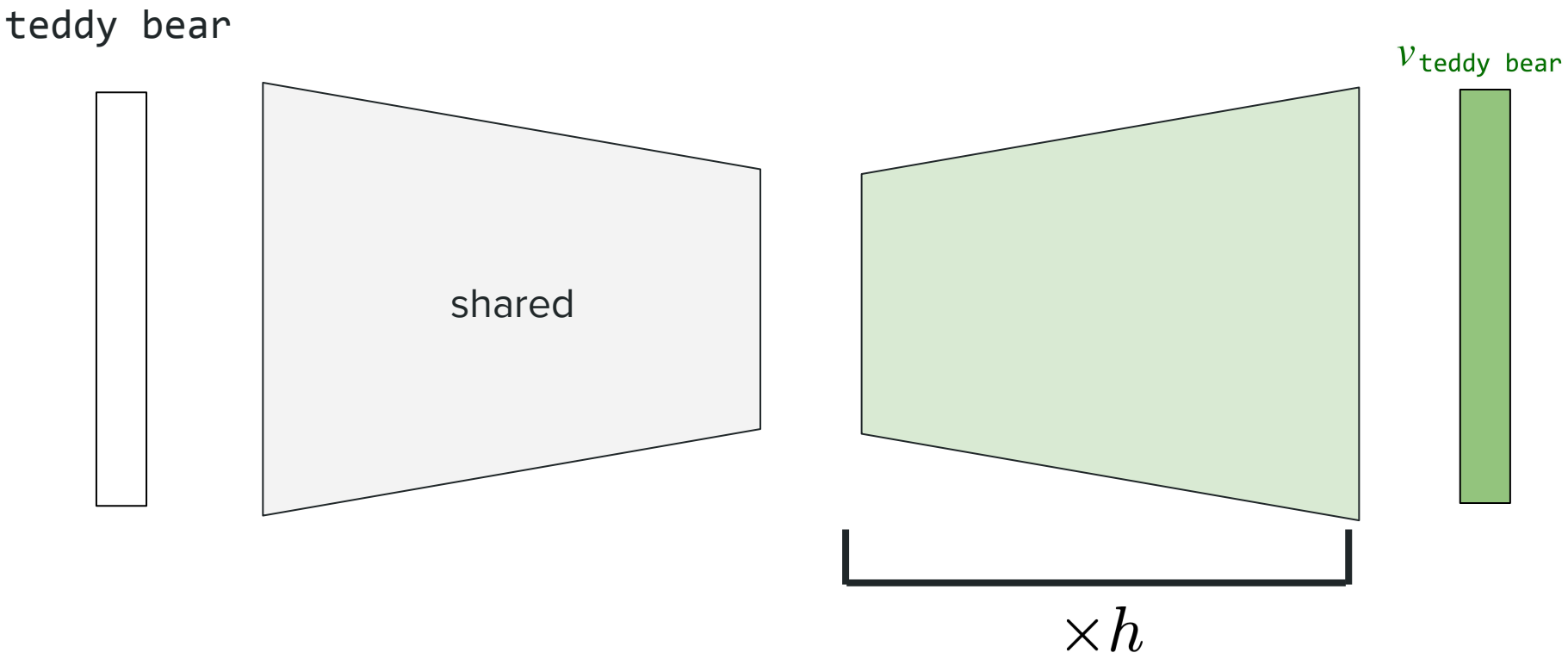
AFTER



DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model, by DeepSeek, 2024.

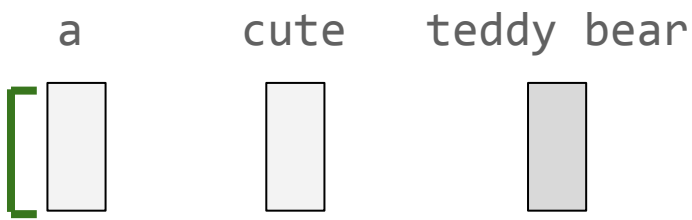
Dimension reduction

AFTER



DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model, by DeepSeek, 2024.

Dimension reduction

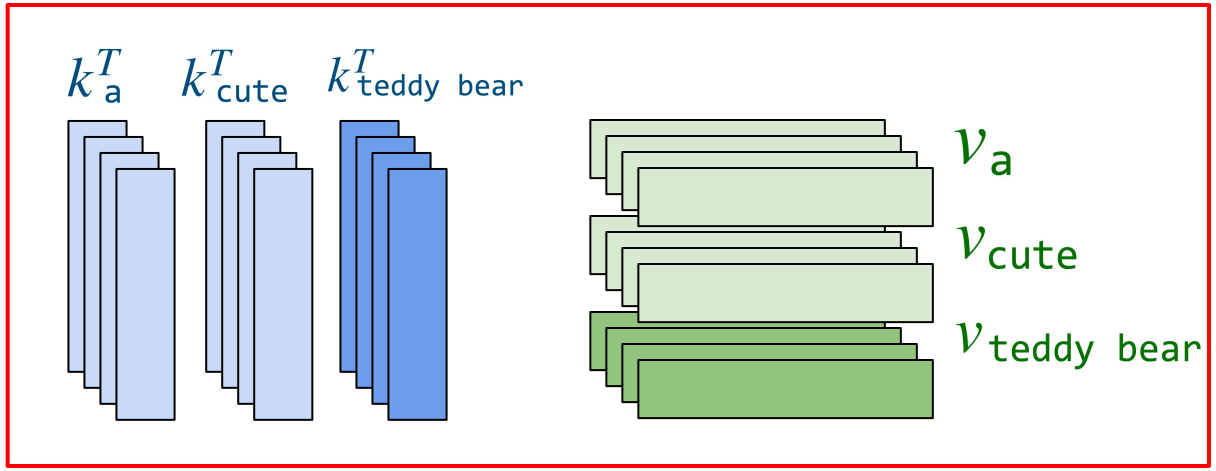


✅ Memory management with PagedAttention

✅ Fewer vectors with GQA

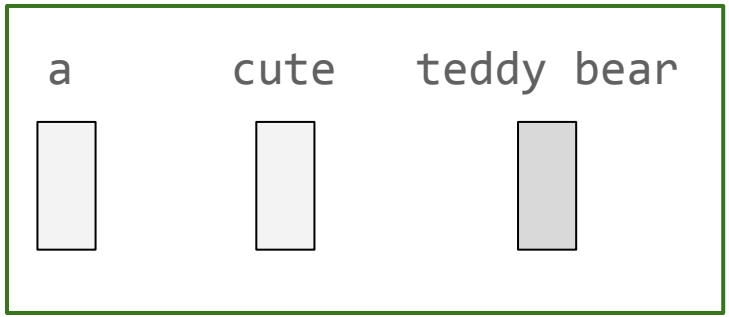
✅ Compressed vectors with MLA

Summary: before



- ❌ Vectors fragmented
- ❌ Too many vectors
- ❌ Vectors too long

Summary: after



✅ Memory management with PagedAttention

✅ Fewer vectors with GQA

✅ Compressed vectors with MLA

Optimizing next token generation

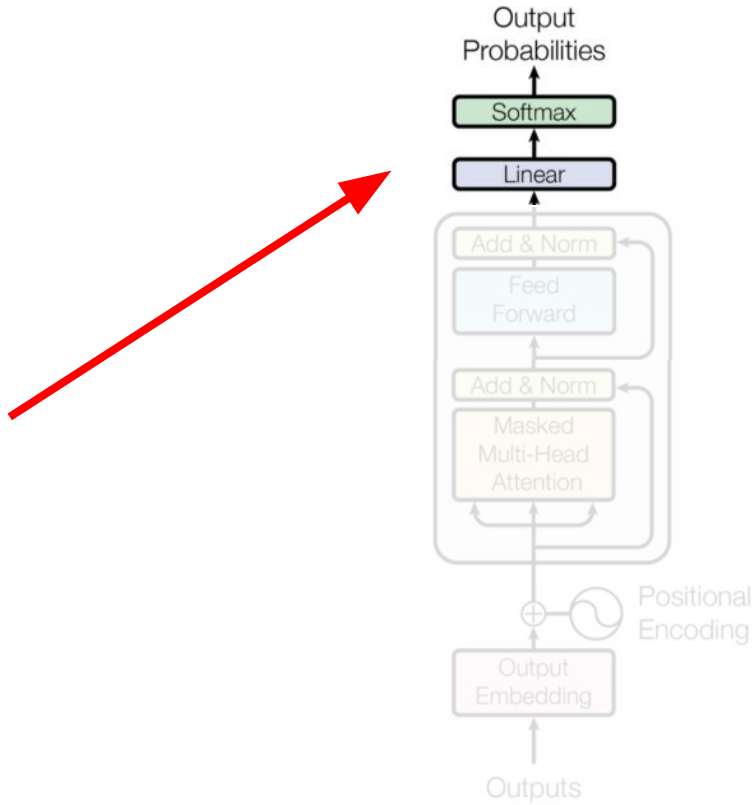
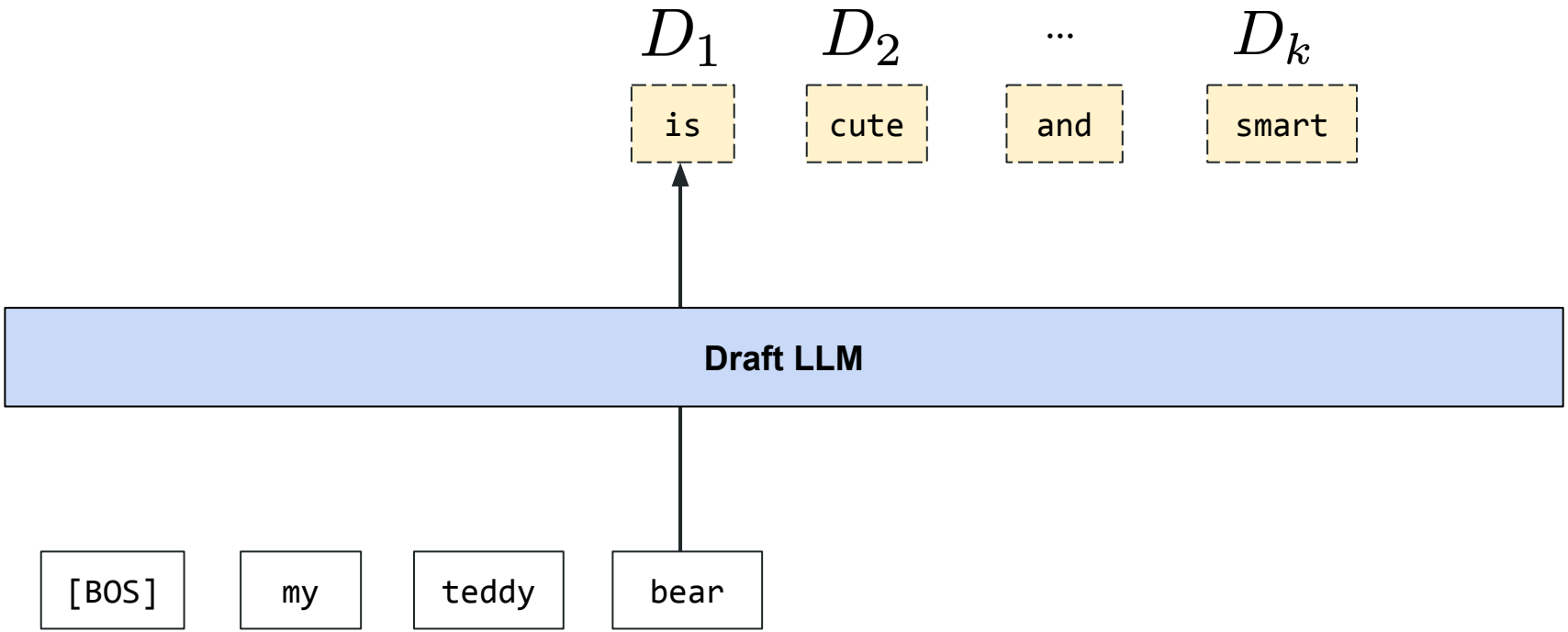


Figure adapted from Attention Is All You Need, Vaswani et al., 2017.

Speeding up inference with speculative decoding

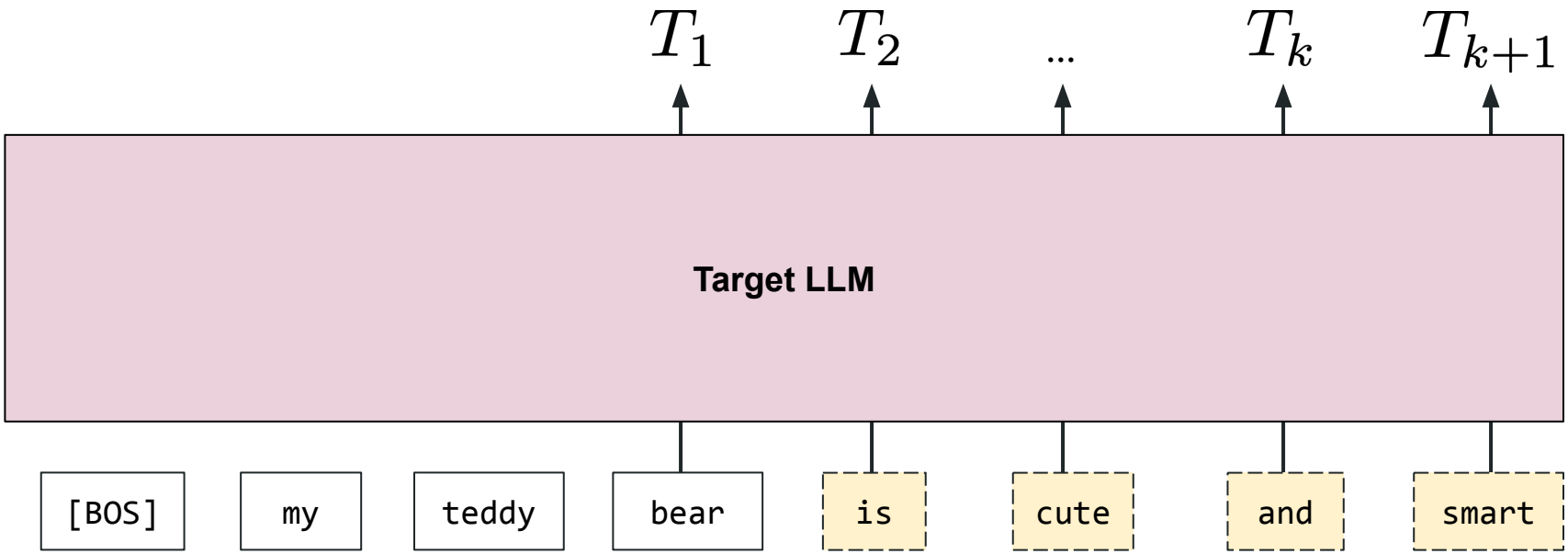
Idea. Use a draft model to generate tokens...



Accelerating Large Language Model Decoding with Speculative Sampling, Chen et al., 2023.

Speeding up inference with speculative decoding

Idea. Use a draft model to generate tokens... validated by a target model!



Accelerating Large Language Model Decoding with Speculative Sampling, Chen et al., 2023.

Speeding up inference with speculative decoding

- If Target LLM is more confident than the Draft LLM



Accelerating Large Language Model Decoding with Speculative Sampling, Chen et al., 2023.

Speeding up inference with speculative decoding

- If Target LLM is more confident than the Draft LLM

- Otherwise
  - with the ratio of probabilities



  - with remaining probability



Speeding up inference with speculative decoding

- If Target LLM is more confident than the Draft LLM

- Otherwise
  - with the ratio of probabilities

  - with remaining probability

If a rejection happens, re-sample next token with residual distribution and exit

Accelerating Large Language Model Decoding with Speculative Sampling, Chen et al., 2023.

Speeding up inference with speculative decoding

Sampling algorithm. We distinguish the following cases:

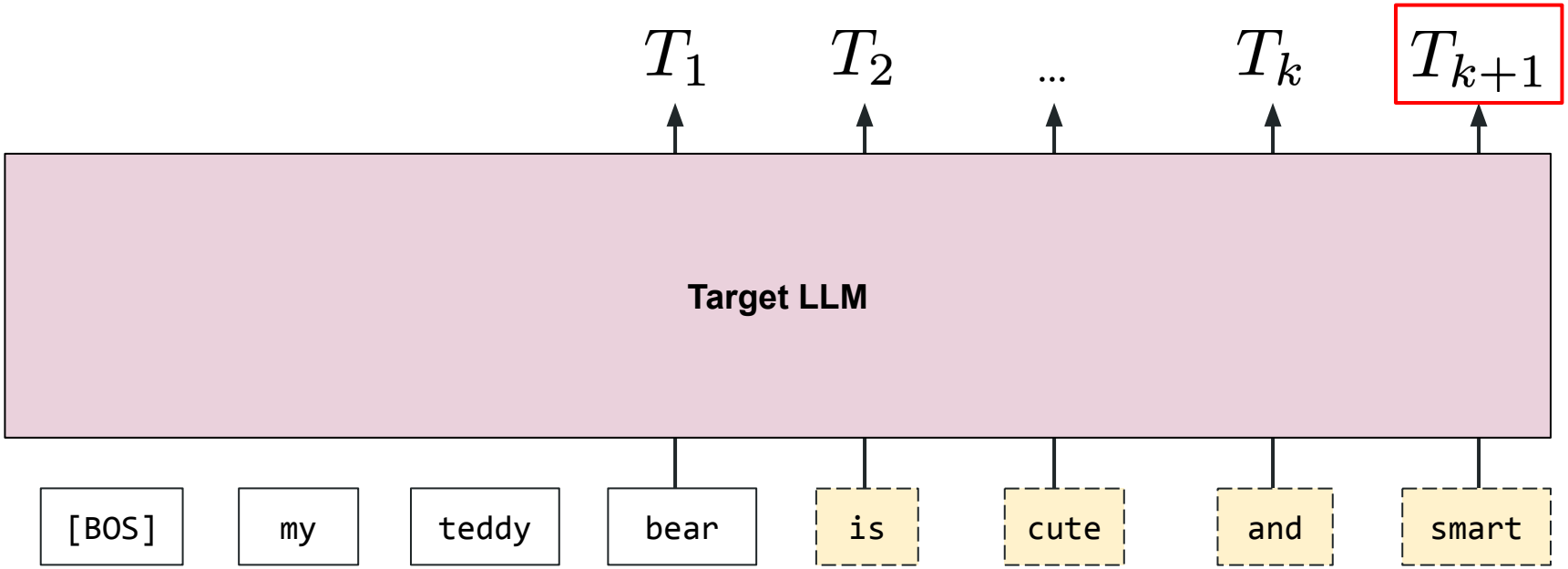
- If $T_i(\boxed{\texttt{token}}) \geqslant D_i(\boxed{\texttt{token}})$ → $\boxed{\texttt{token}}$ ✅
- Otherwise
  - with probability $T_i(\boxed{\texttt{token}})/D_i(\boxed{\texttt{token}})$ → $\boxed{\texttt{token}}$ ✅
  - with probability $1 - T_i(\boxed{\texttt{token}})/D_i(\boxed{\texttt{token}})$ → $\boxed{\texttt{token}}$ ❌

If a rejection happens, re-sample next token with distribution $[T_i - D_i]_+$ (normalized) and exit

Accelerating Large Language Model Decoding with Speculative Sampling, Chen et al., 2023.

Speeding up inference with speculative decoding

Bonus. Next token for free!



Accelerating Large Language Model Decoding with Speculative Sampling, Chen et al., 2023.

Speeding up inference with speculative decoding

Bonus. Next token for free!

Remarks. To work well:

- Draft LLM should be noticeably faster than Target LLM

Accelerating Large Language Model Decoding with Speculative Sampling, Chen et al., 2023.

Speeding up inference with speculative decoding

Bonus. Next token for free!

Remarks. To work well:

- Draft LLM should be noticeably faster than Target LLM
- Draft LLM's next-token distribution should align with the Target LLM's

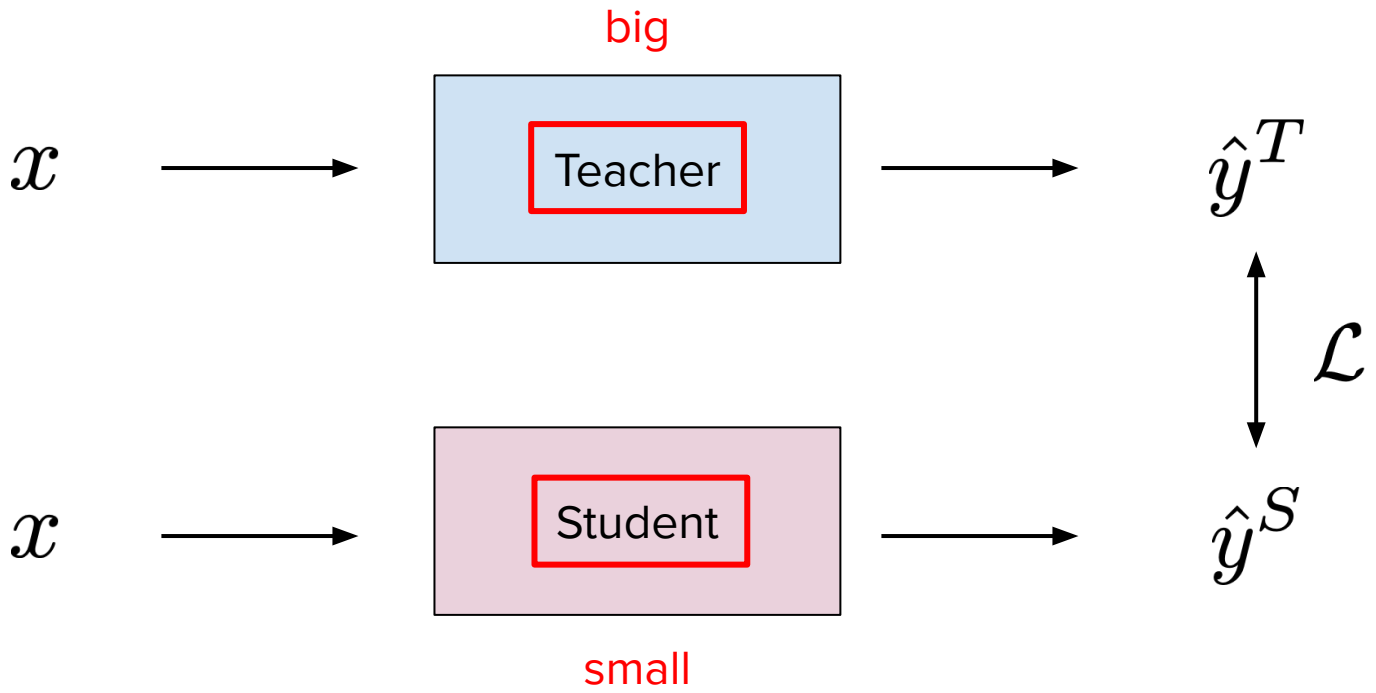


knowledge distillation!

DistillSpec: Improving Speculative Decoding via Knowledge Distillation, Zhou et al., 2023

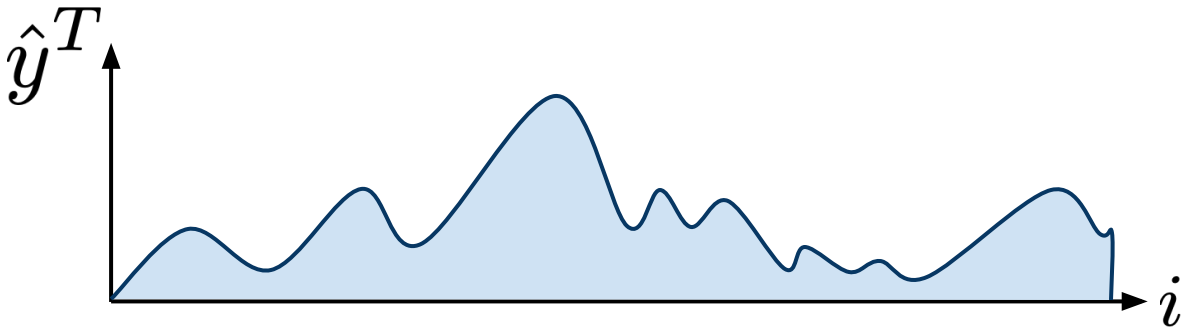
Knowledge distillation

Idea. Learn what the teacher knows.



Distilling the Knowledge in a Neural Network, Hinton et al., 2015.

Knowledge distillation



$$\mathrm{KL}(\hat{y}^T \| \hat{y}^S) = \sum_i \hat{y}^T_{(i)} \log \left(\frac{\hat{y}^T_{(i)}}{\hat{y}^S_{(i)}} \right)$$

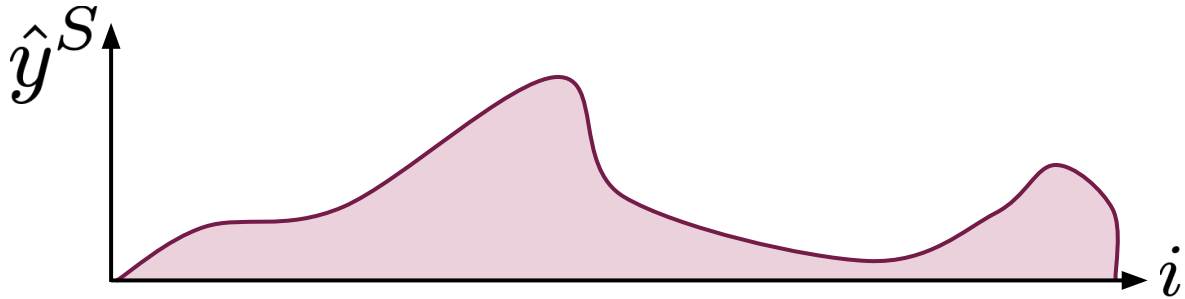
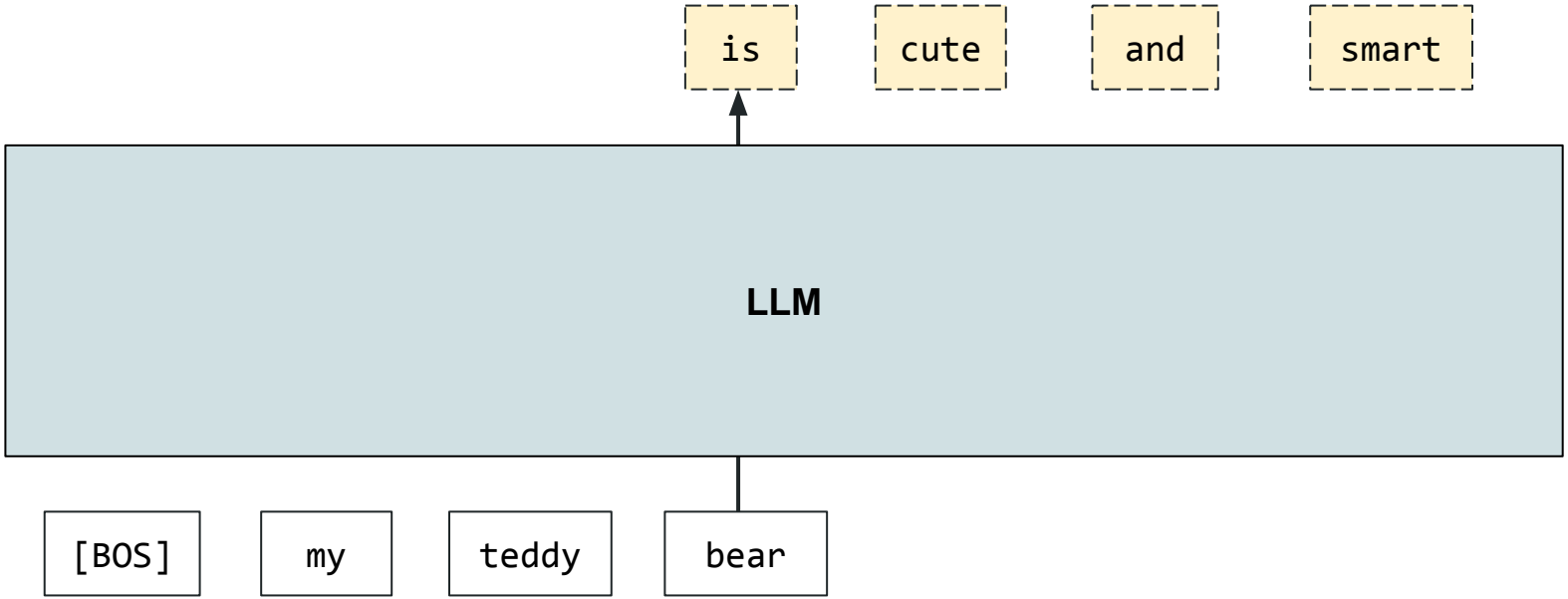


Figure adapted from Super Study Guide: Transformers & Large Language Models, Amidi et al., 2024.

Generate several tokens at once via MTP

MTP = **M**ulti-**T**oken **P**rediction



Better & Faster Large Language Models via Multi-token Prediction, Gloeckle et al., 2024.

Generate several tokens at once via MTP

Idea. Train k prediction heads: same draft and target models!

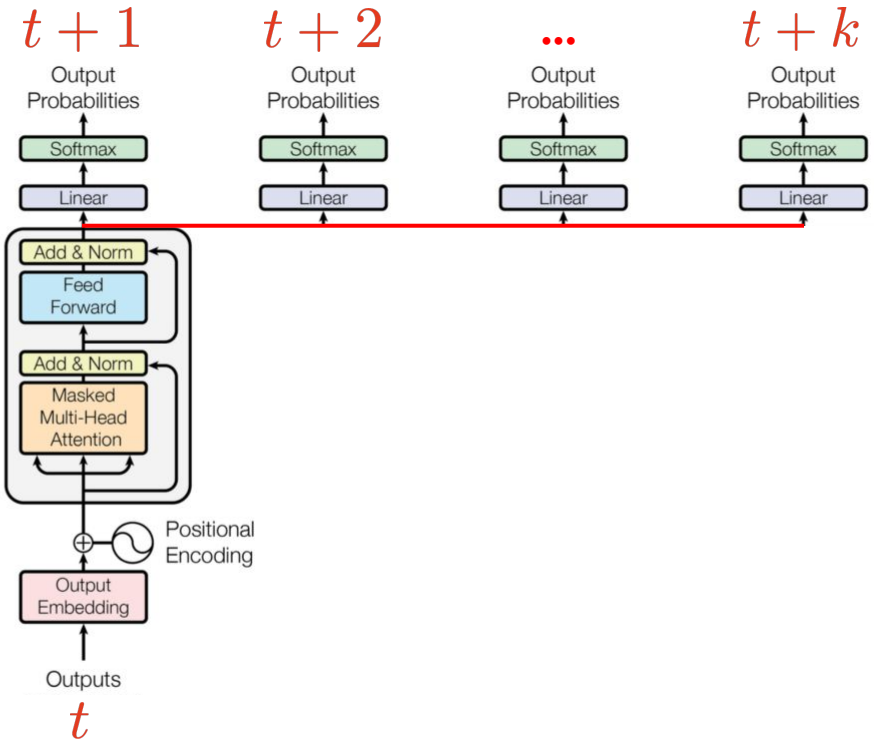


Figure adapted from Attention Is All You Need, Vaswani et al., 2017.



Transformers & Large Language Models

Motivation

Flash attention

Distributed training

Inference optimizations

Discussion

Takeaways



How do we make it **fit**?

- Organize vectors better (**PagedAttention**)
- Fewer and smaller vectors (**GQA**, **MLA**)

Takeaways

How do we make it **fit**?

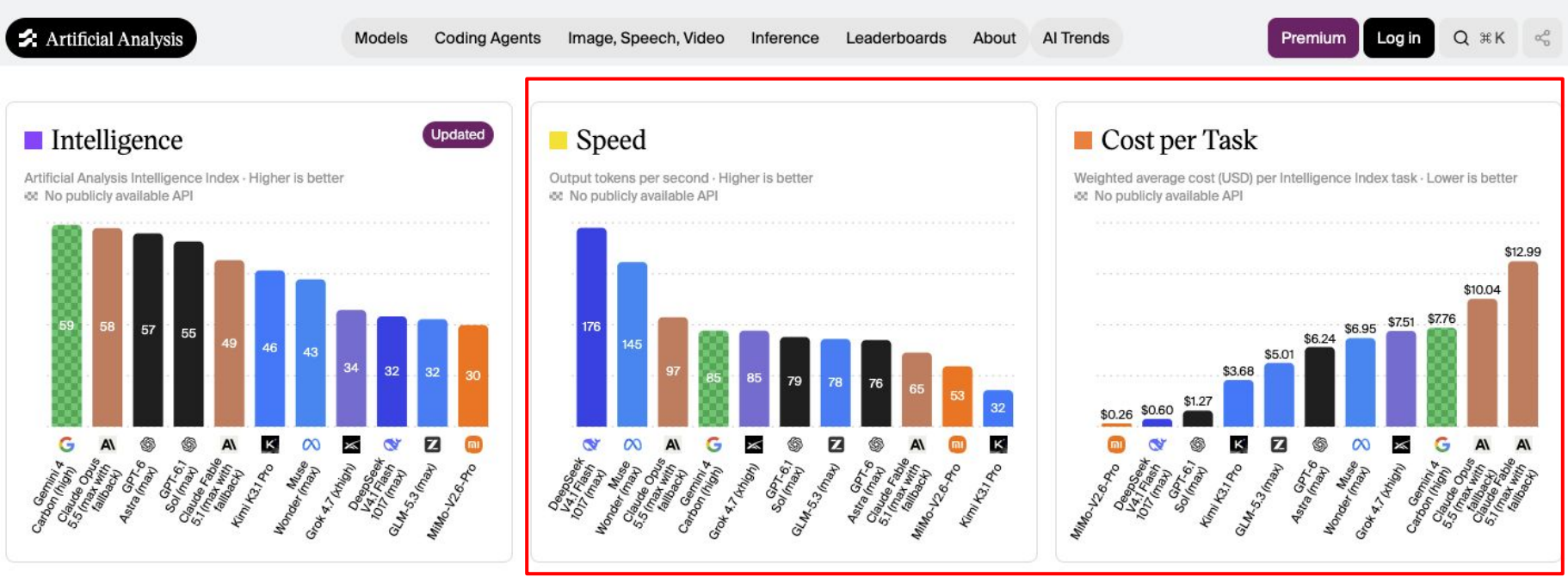
- Organize vectors better (**PagedAttention**)
- Fewer and smaller vectors (**GQA**, **MLA**)



How do we make it **fast**?

- Generate more tokens at once (**Speculative decoding**, **MTP**)

Optimizations: active area of research!



Screenshot taken from the Artificial Analysis homepage on October 29, 2026.

Thank you for your attention!